

Slide 39 Multicast

- ❑ A **process group** consists of several processes
 - A process might belong to **multiple groups**
- ❑ A **multicast** operation sends a message to all the members in a group.
- ❑ The **aim** of a multicast operations is, for each process in a group, the process should receive a copy of the message sent to the group.
 - Various delivery guarantees might be required.

The meaning of deliver

network

multicast messages

multicast middleware

application

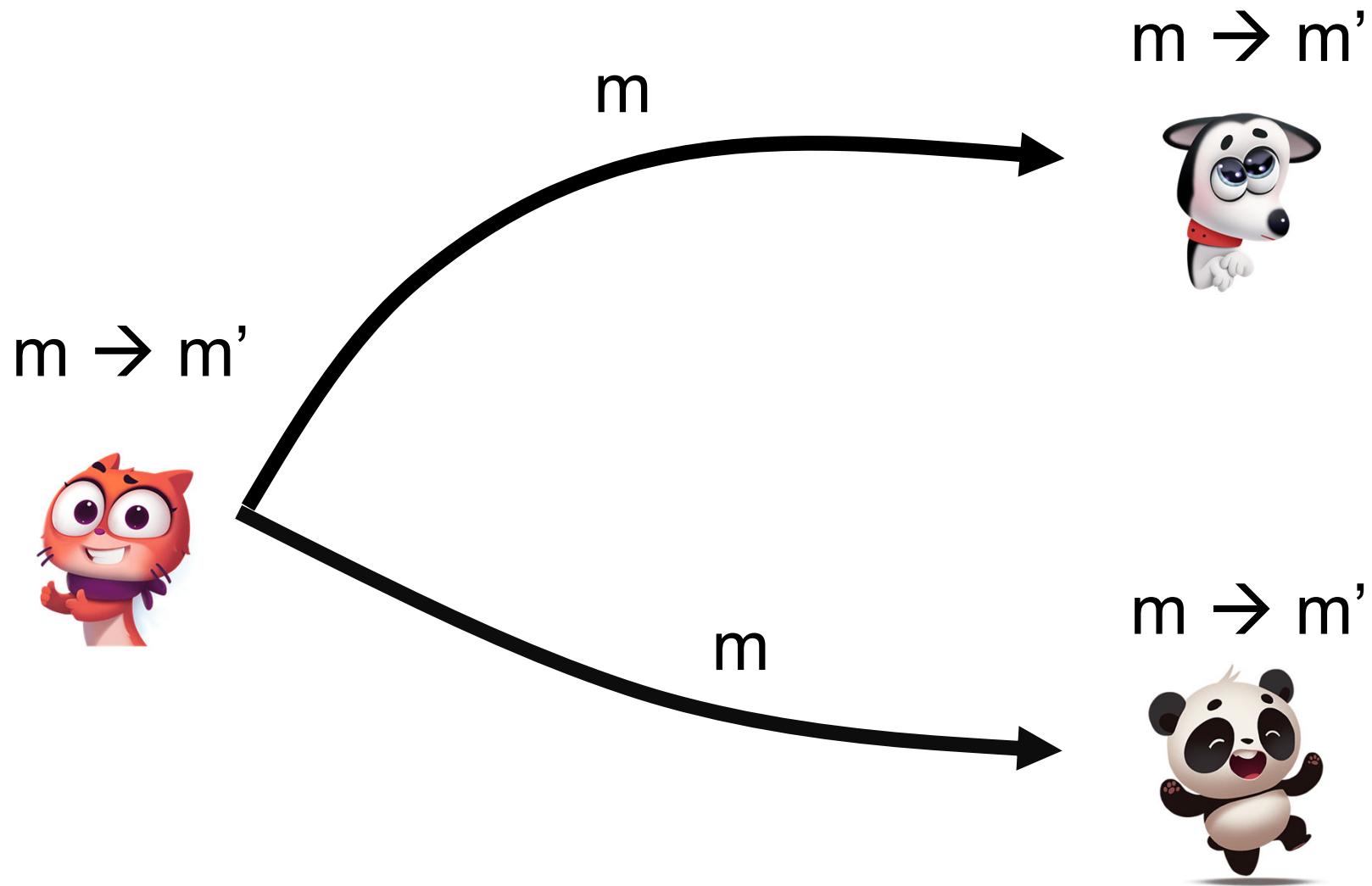
multicast messages

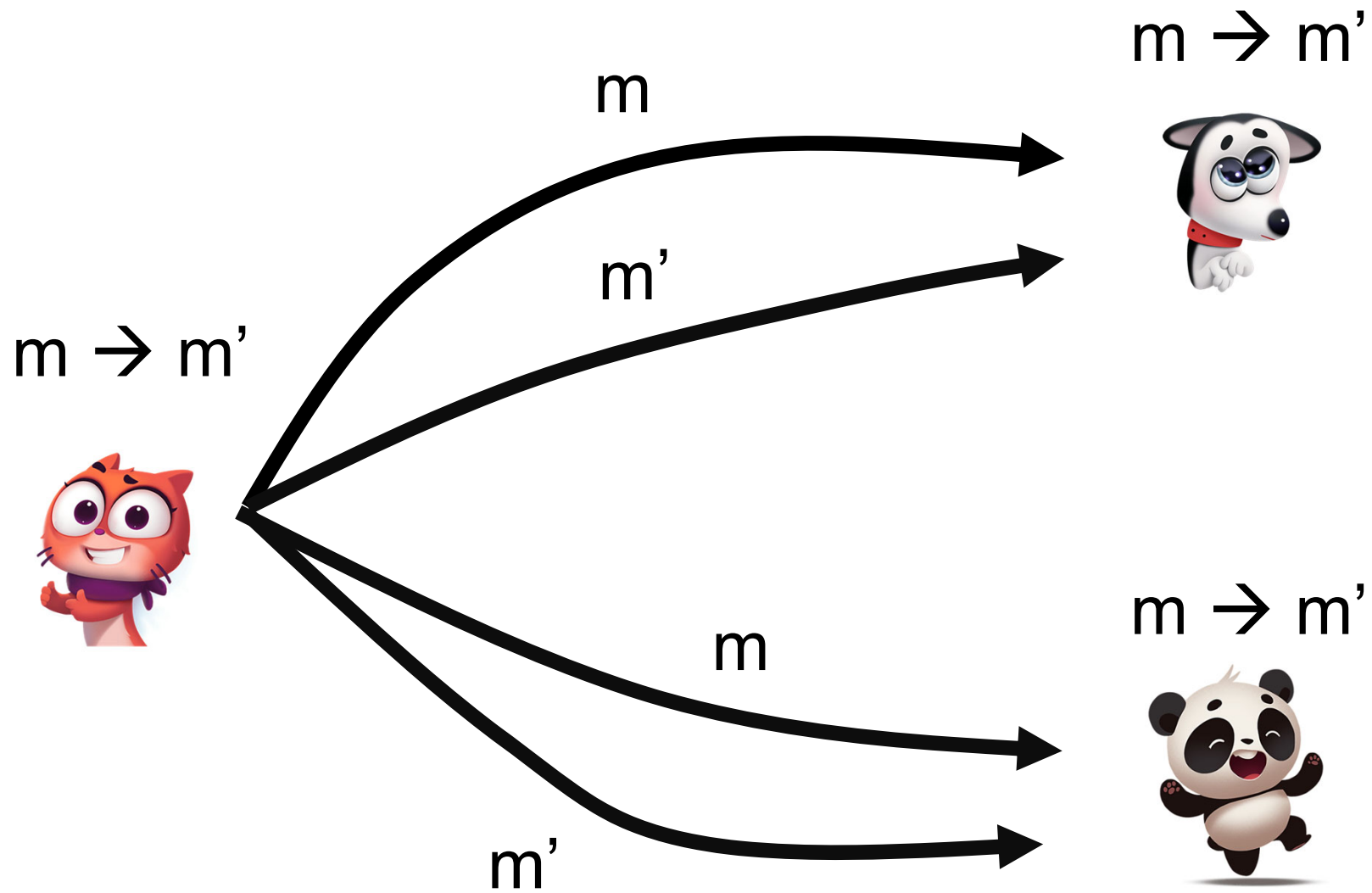
multicast middleware

application

Ordered multicast

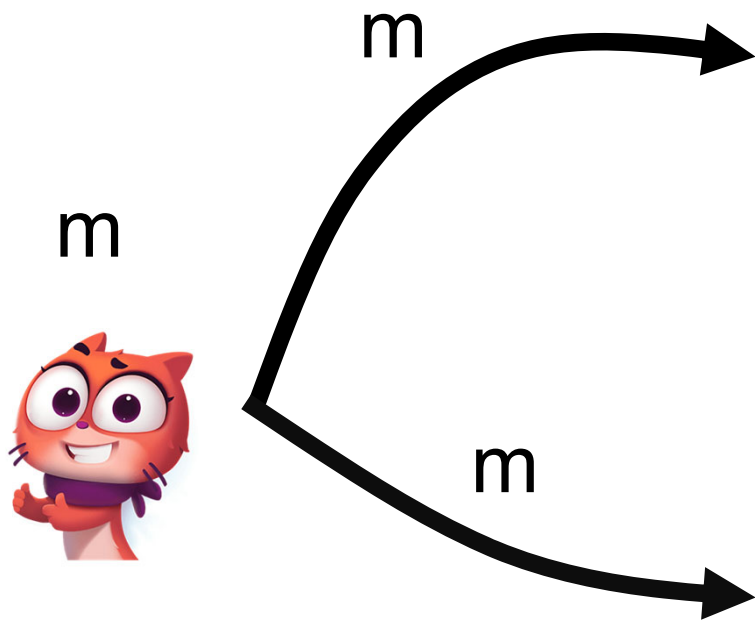
- **FIFO ordering**: if a process multicasts m and then multicasts m' , then all processes in the group that delivers m' will deliver m before m'

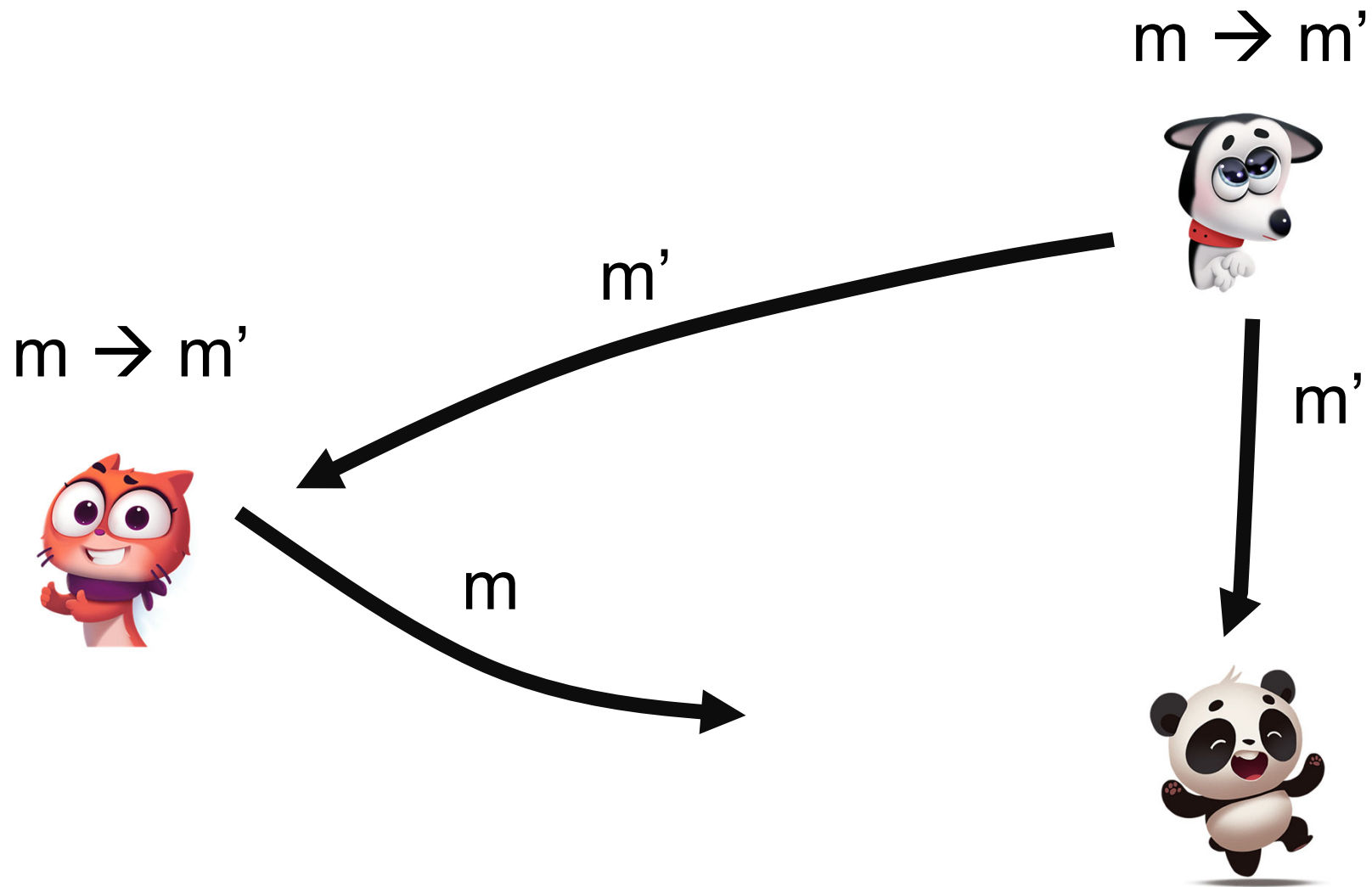




Ordered multicast

- **Causal ordering**: if multicast m happens before multicast m' , then all the processes in the group will deliver m before m'





$$m \rightarrow m'$$



$$m'$$



$$m \rightarrow m'$$



$$m \rightarrow m'$$



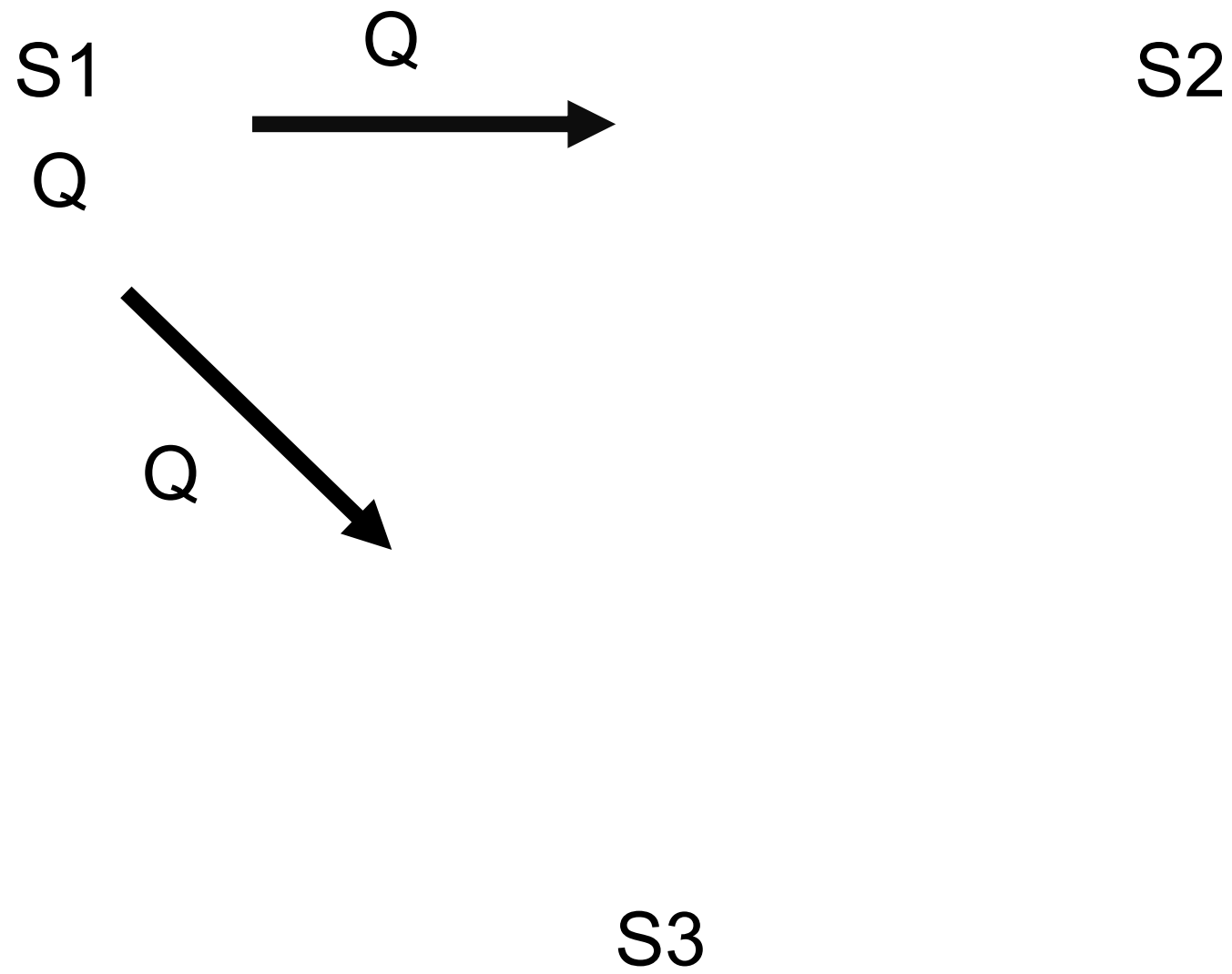
$$m$$

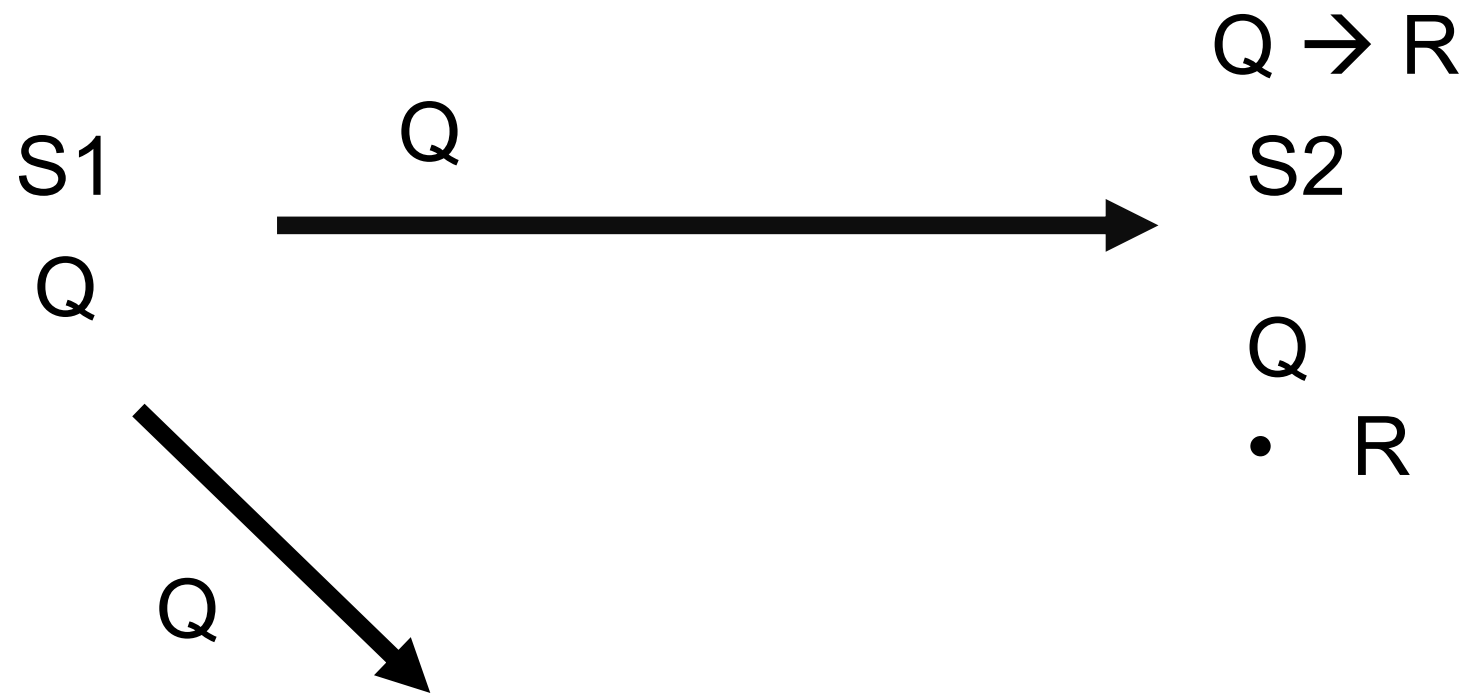


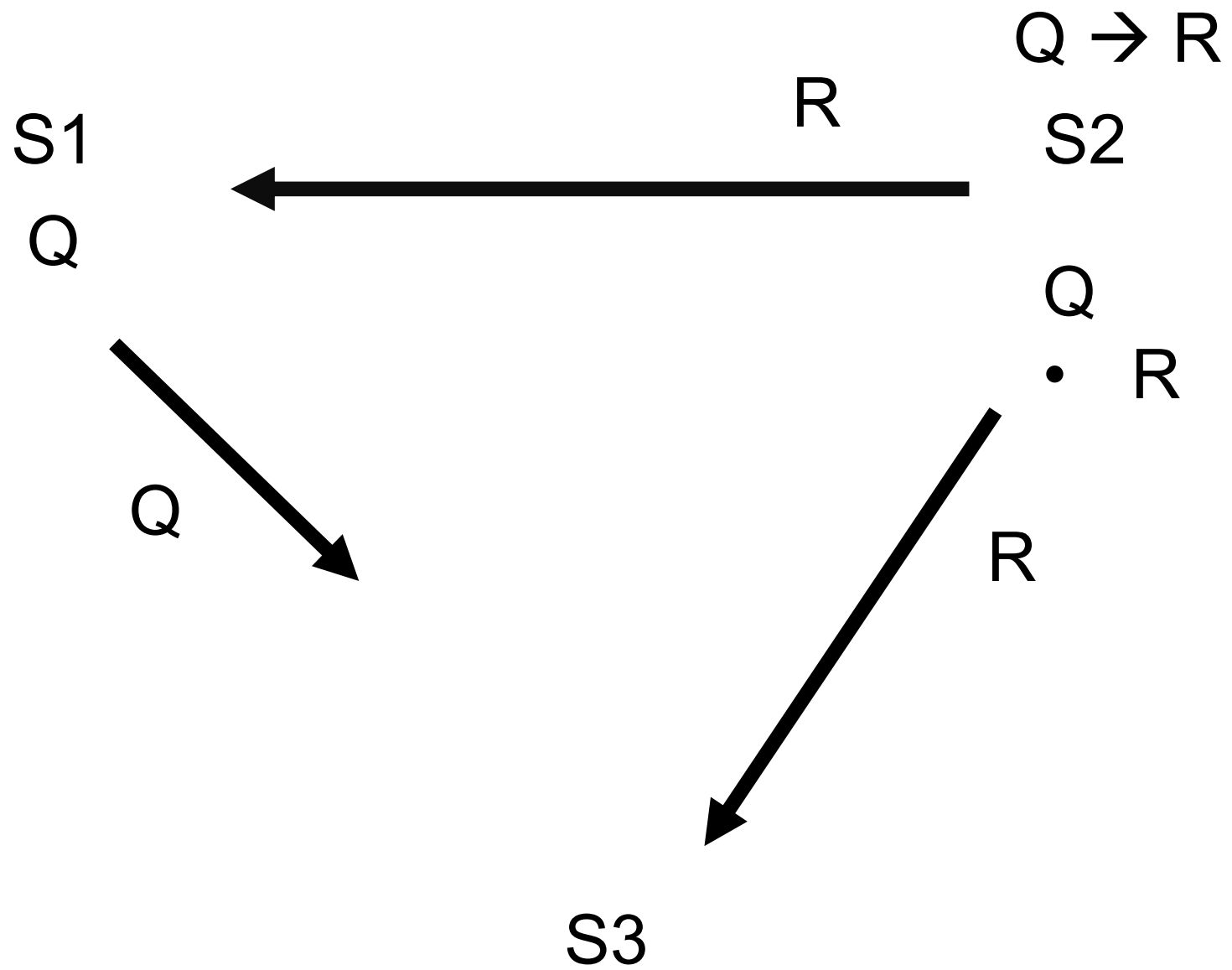
Replicated discussion forum

Question

- Response 1
- Response 2





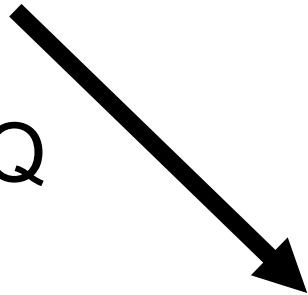


S1

Q

• R

Q



$Q \rightarrow R$

S2

Q

• R

S3

R

Ordered multicast

- **Total ordering**: if a process delivers message m before m' , then all other processes will deliver m before m'



x

S1

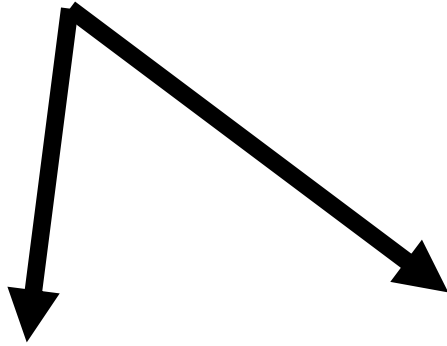


x

S2



$m (x = 1)$

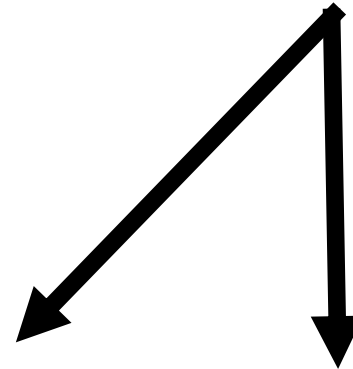


x

$S1$



$m' (x = 2)$



x

$S2$



$m \ (x = 1)$

$x = 1$

S1

$m' \rightarrow m$



$m' \ (x = 2)$

$x = 1$

S2

$m' \rightarrow m$



$m \ (x = 1)$



$m' \ (x = 2)$

$x = 2$

S1

$m \rightarrow m'$

$x = 2$

S2

$m \rightarrow m'$



$m (x = 1)$



$m' (x = 2)$

$x = 2$

S1

$m \rightarrow m'$

$x = 1$

S2

$m' \rightarrow m$

T_1

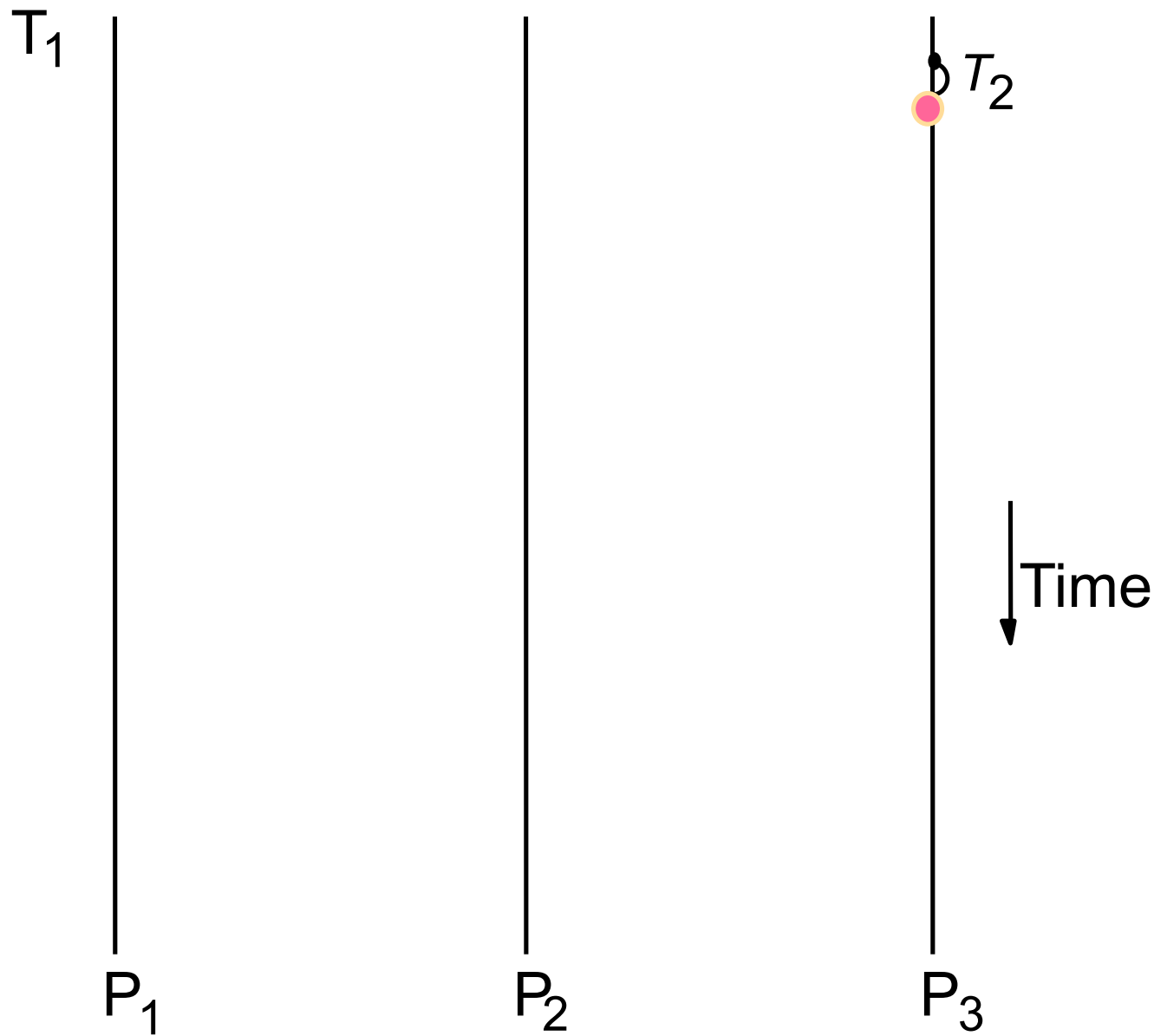
P_1

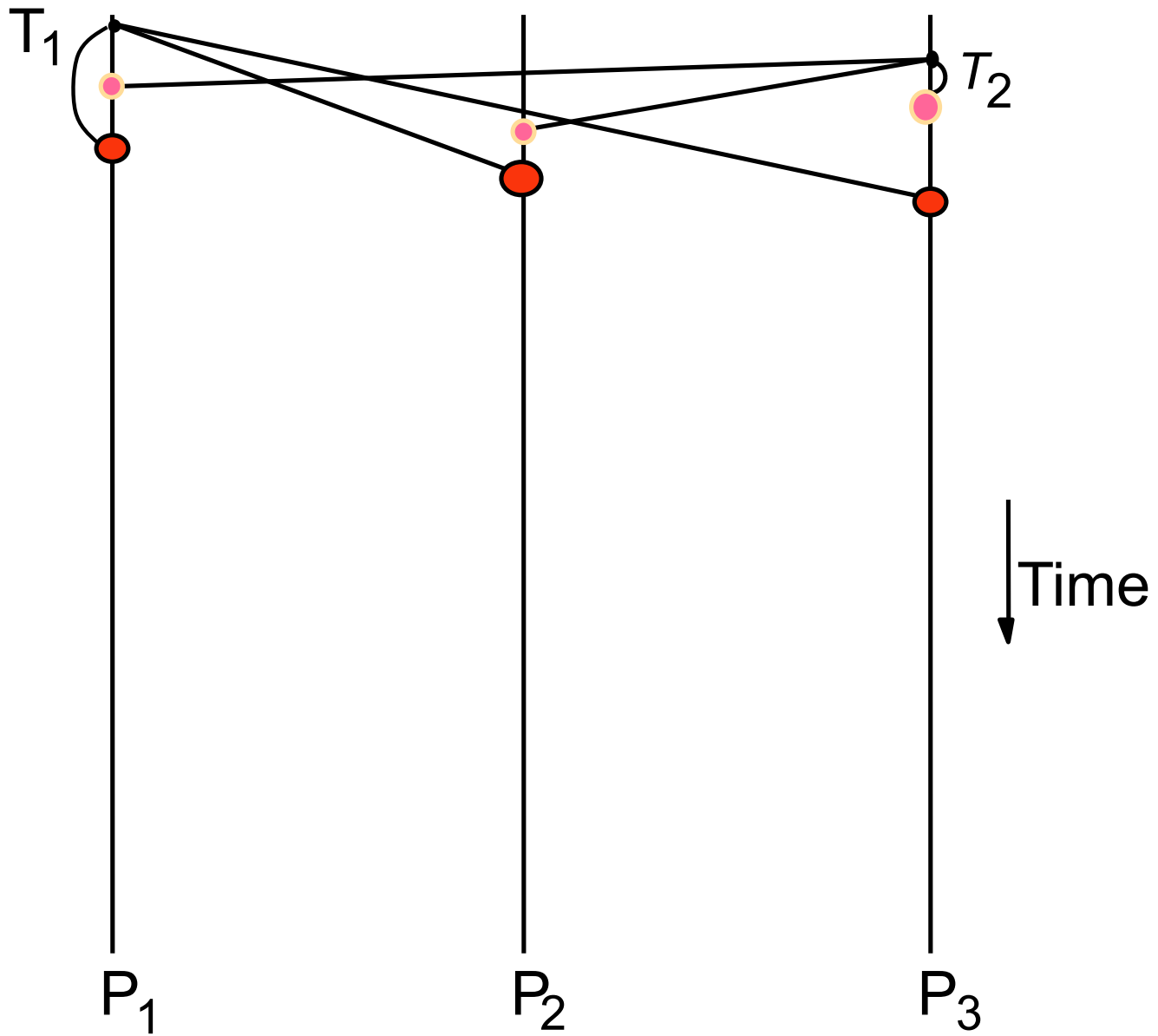
P_2

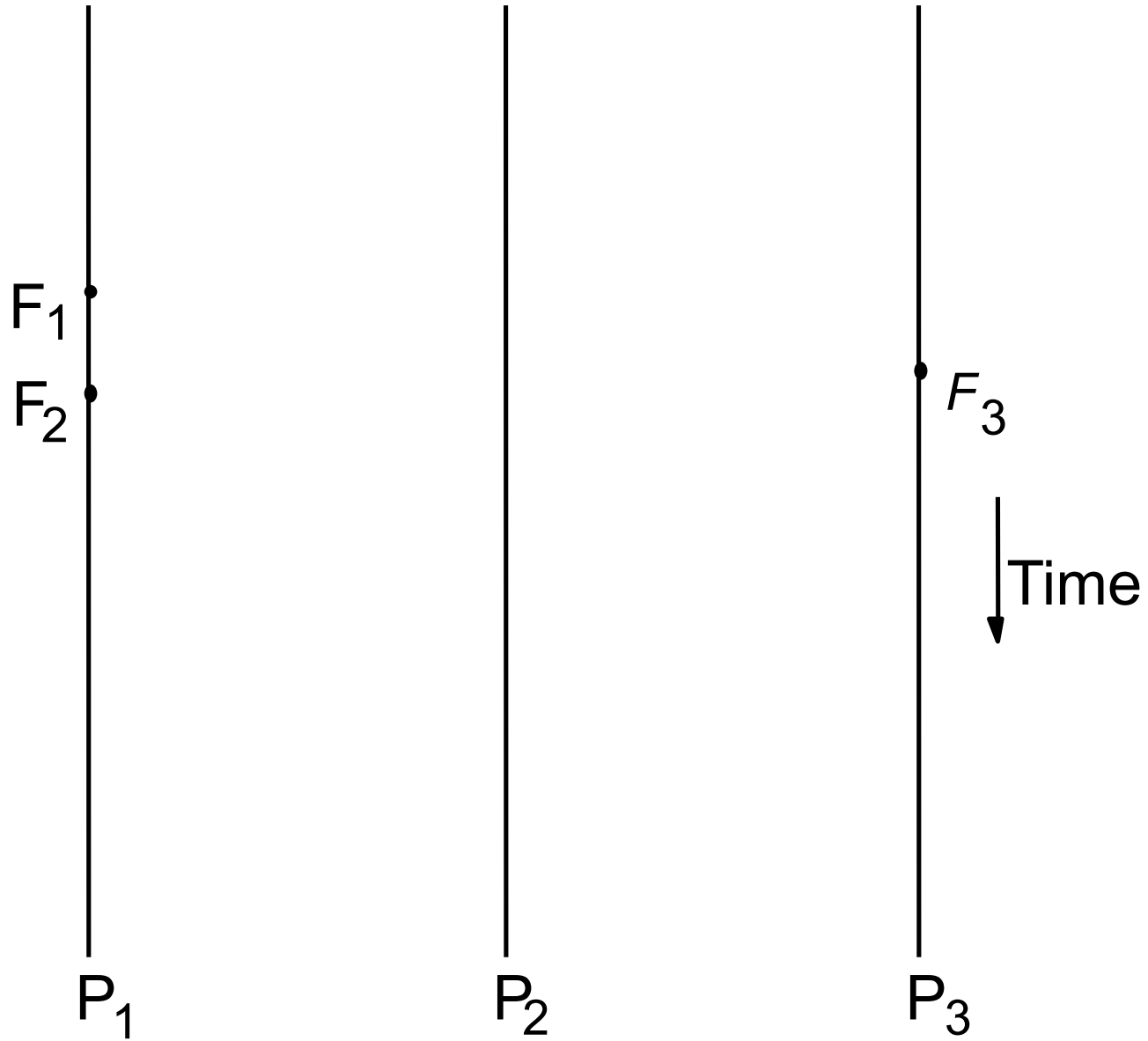
T_2

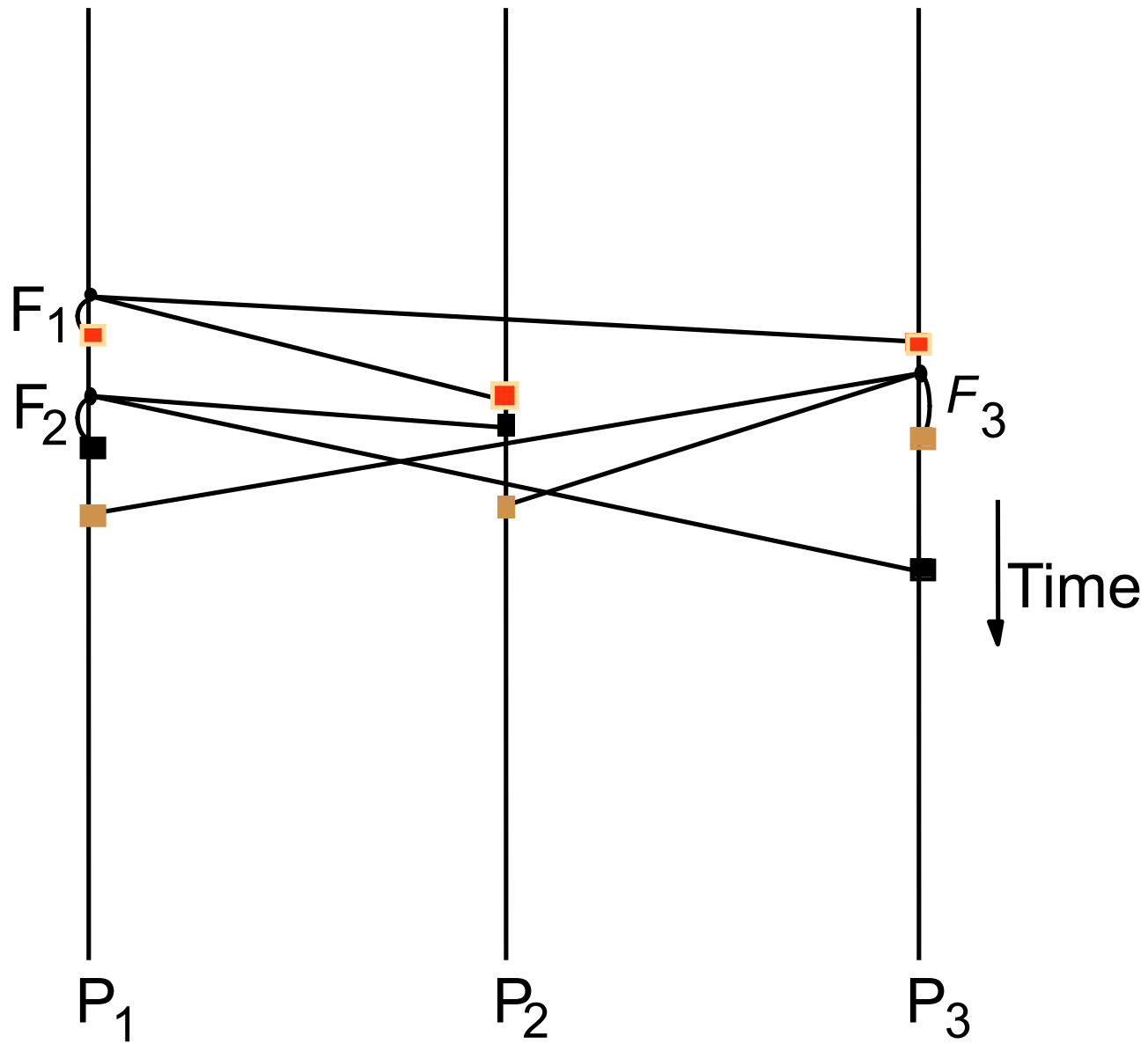
P_3

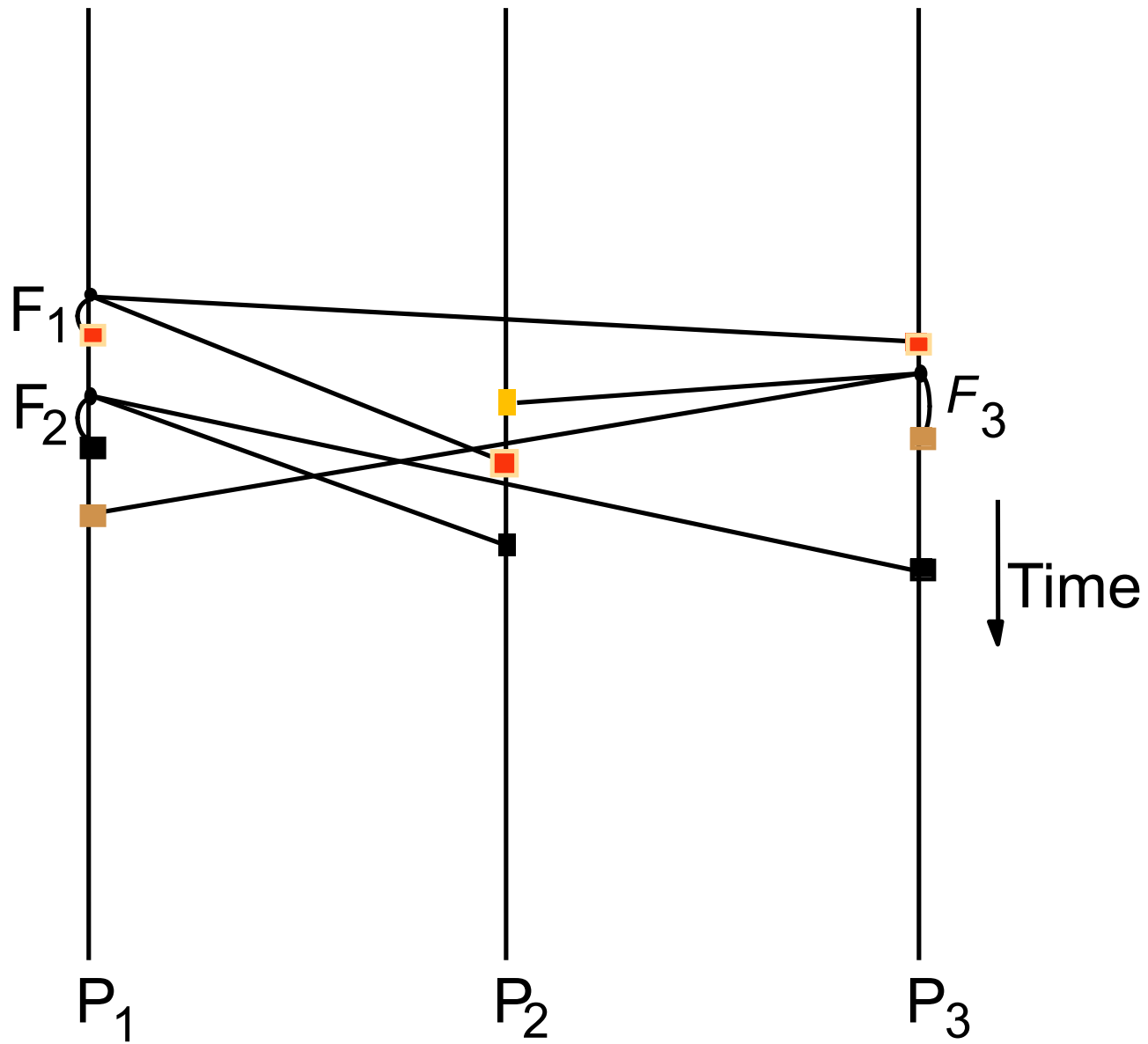
Time

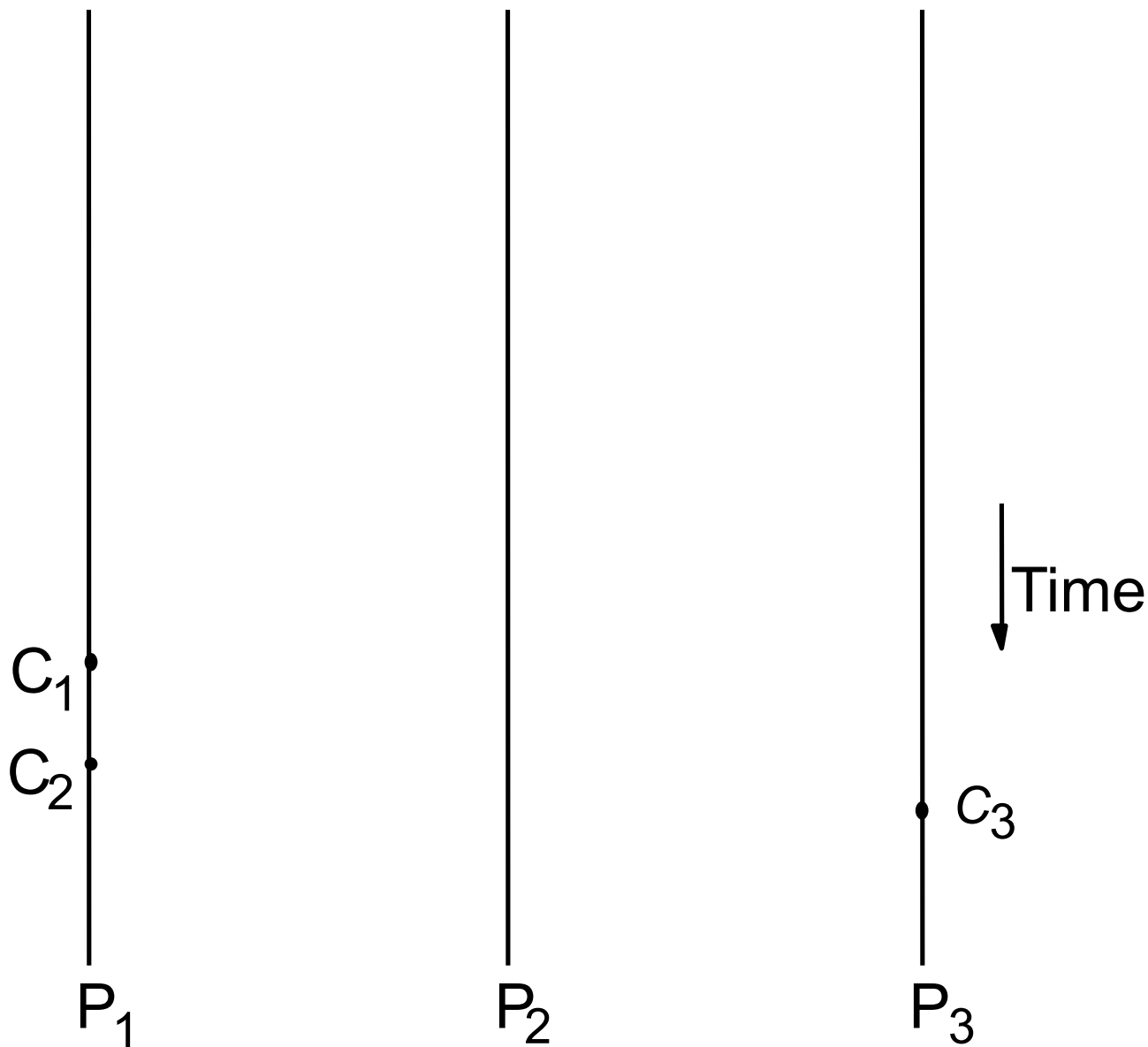


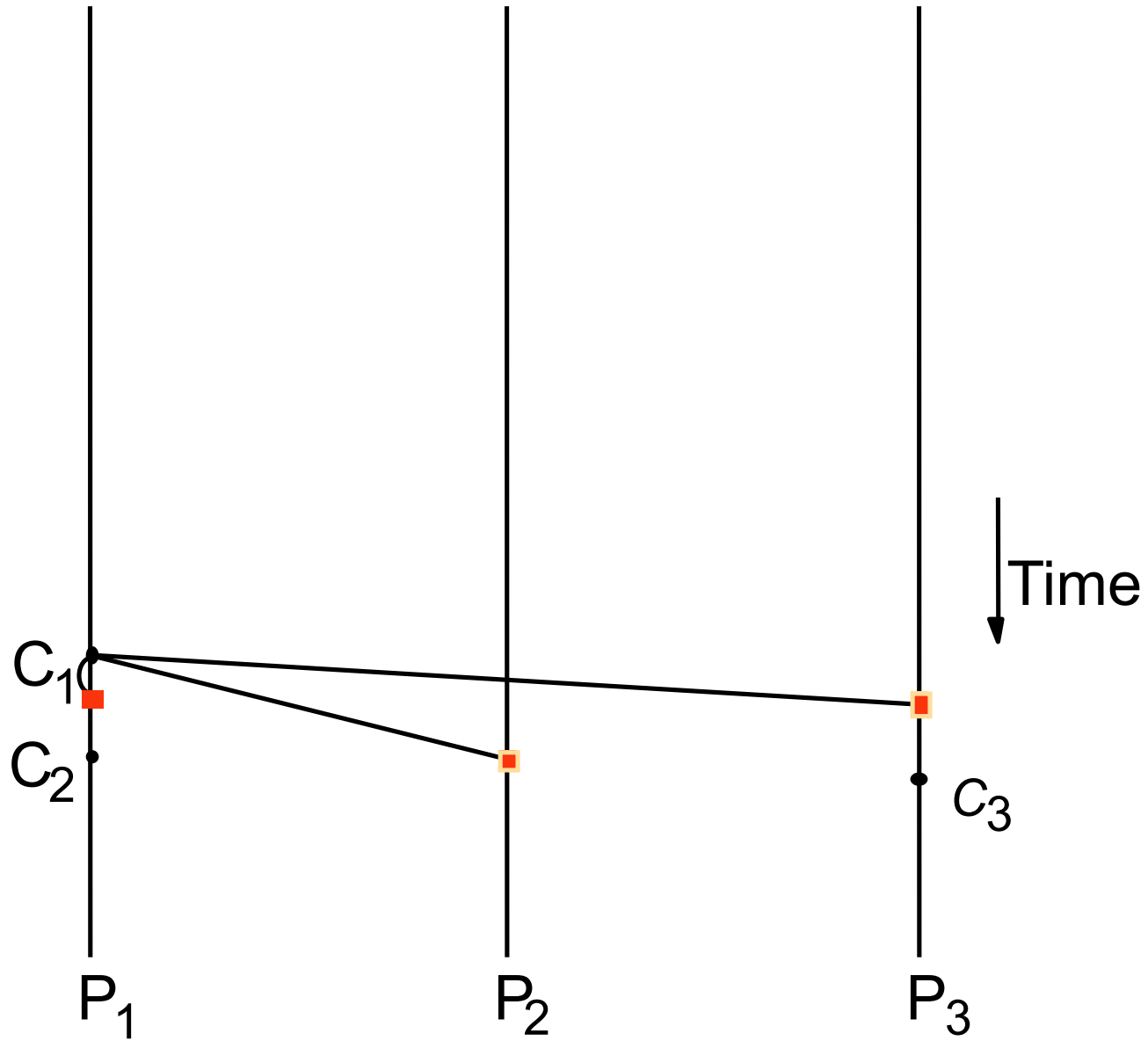


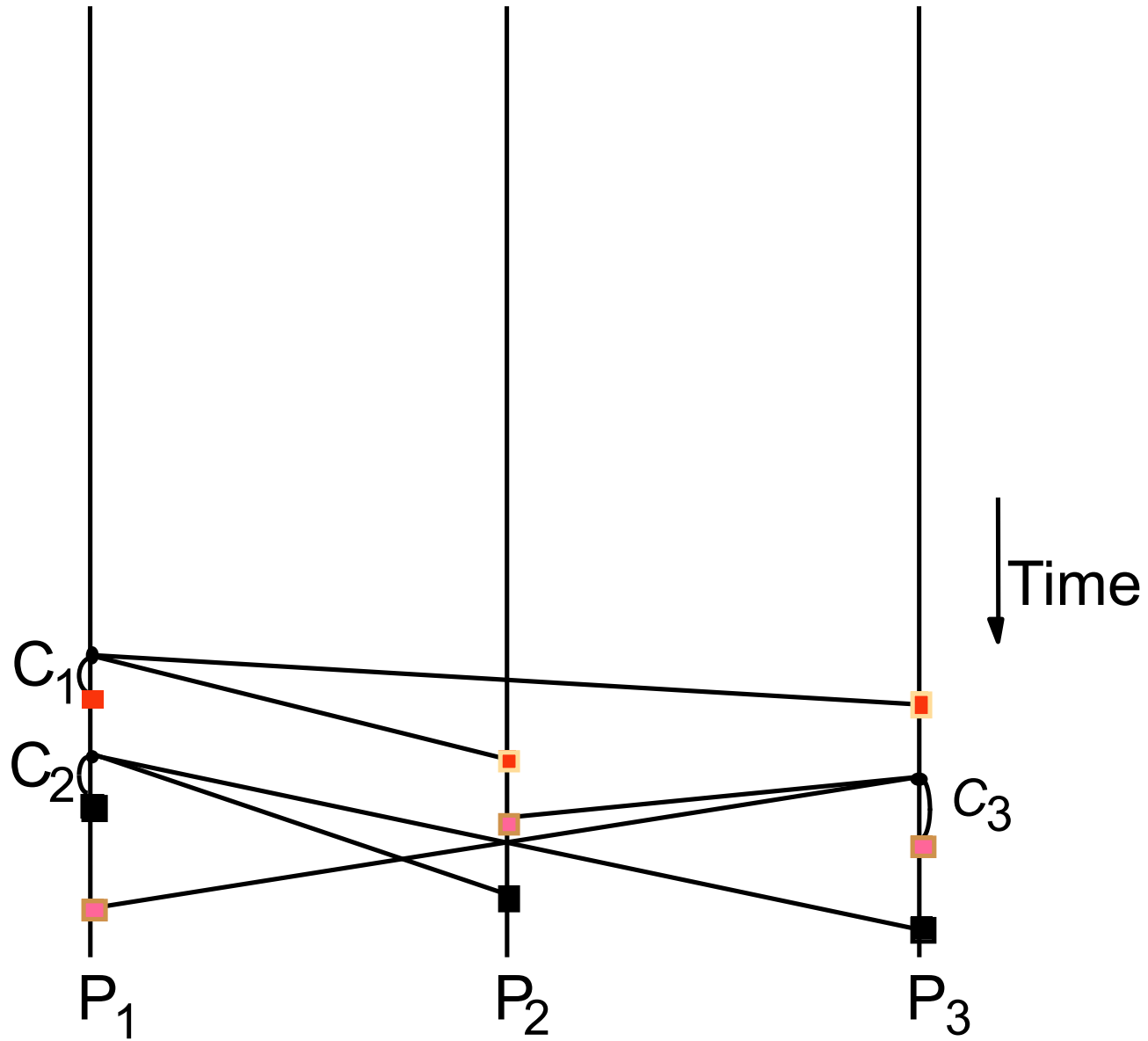












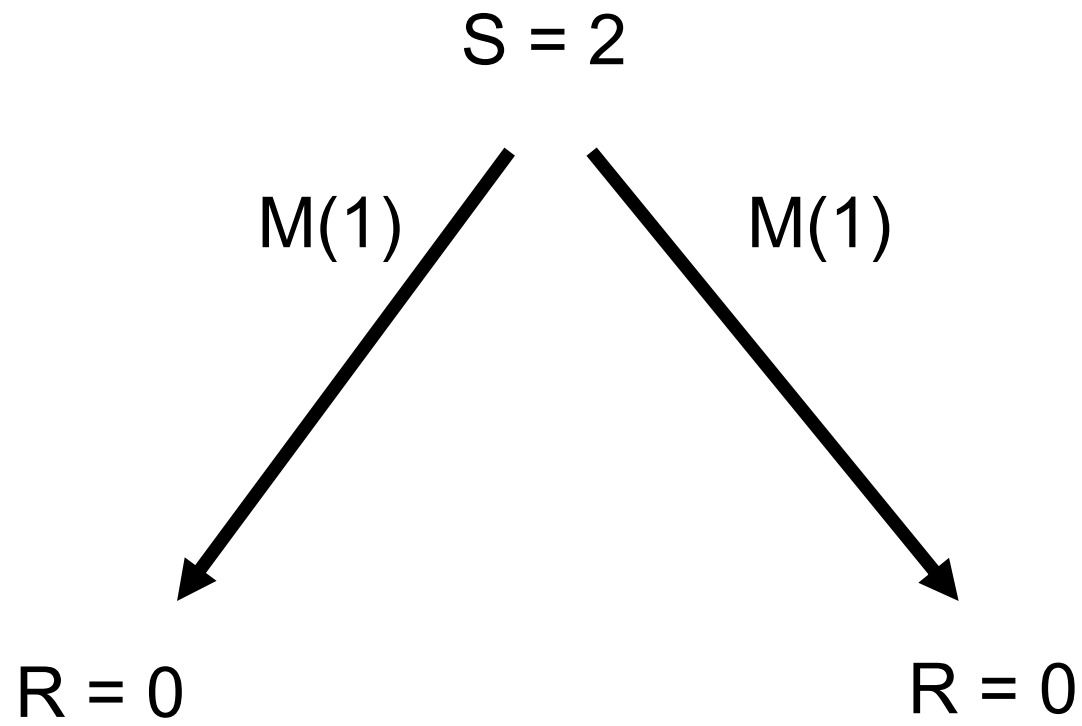
Implementing FIFO ordering

- ❑ A process assigns a sequence number to the message that it broadcasts to the group
- ❑ Each process maintains some variables to record the number of messages that are sent to/received from the other processes
 - S is the sequence number assigned to the broadcast message
 - R_q is the sequence number of the last message that is broadcast by process q
- ❑ When broadcast a message:
 - Assign S as the sequence number of the message
 - Increment S
- ❑ When receives a message from process q
 - If the sequence number of the message is $R_q + 1$, deliver the message.
 - Otherwise, place the message in a waiting queue.
 - The message will be delivered after all the preceding messages are delivered.

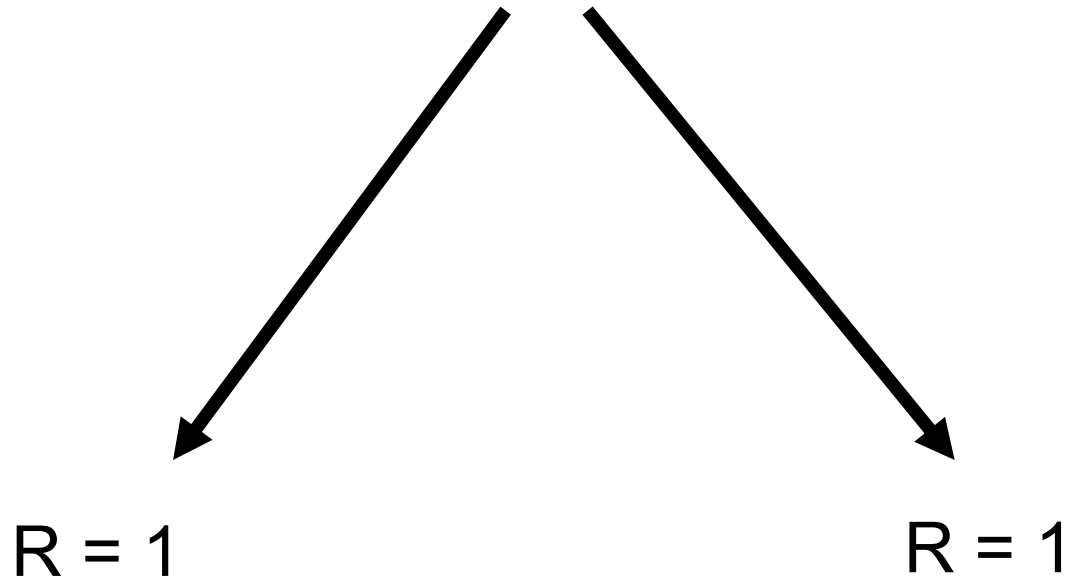
$$S = 1$$

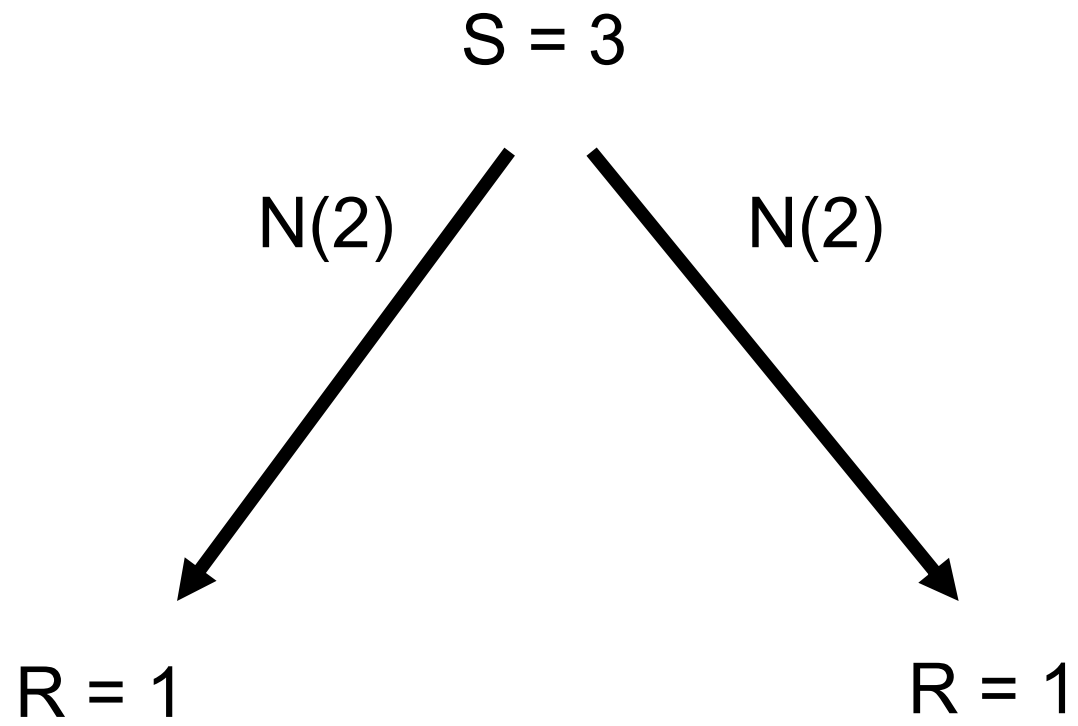
$$R = 0$$

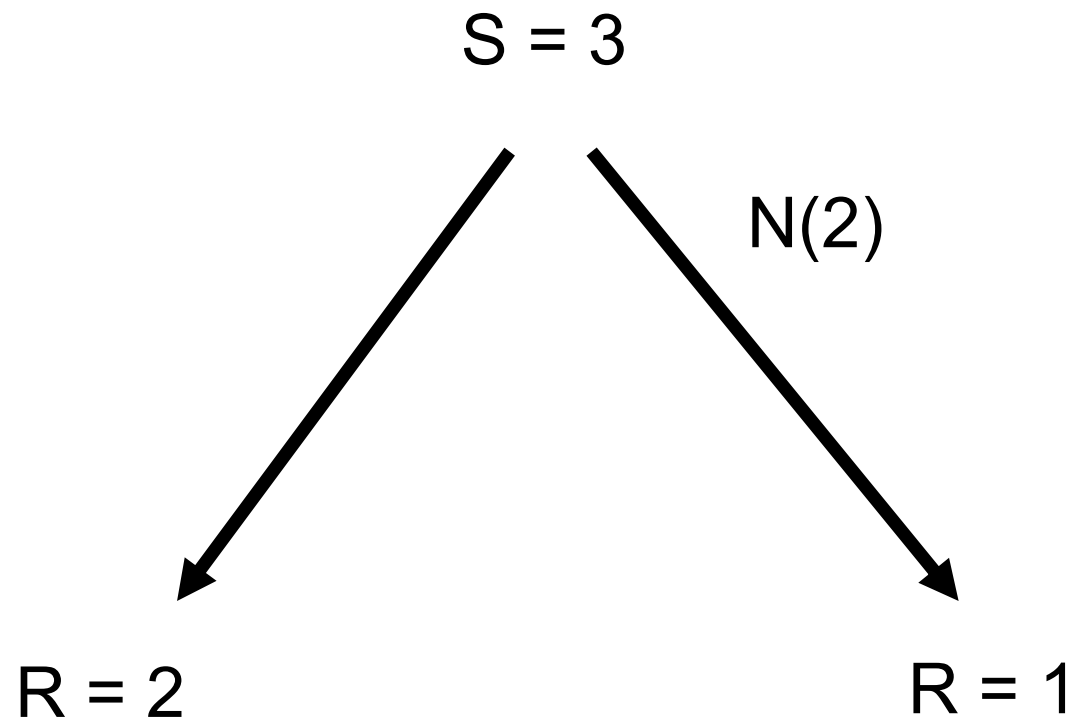
$$R = 0$$

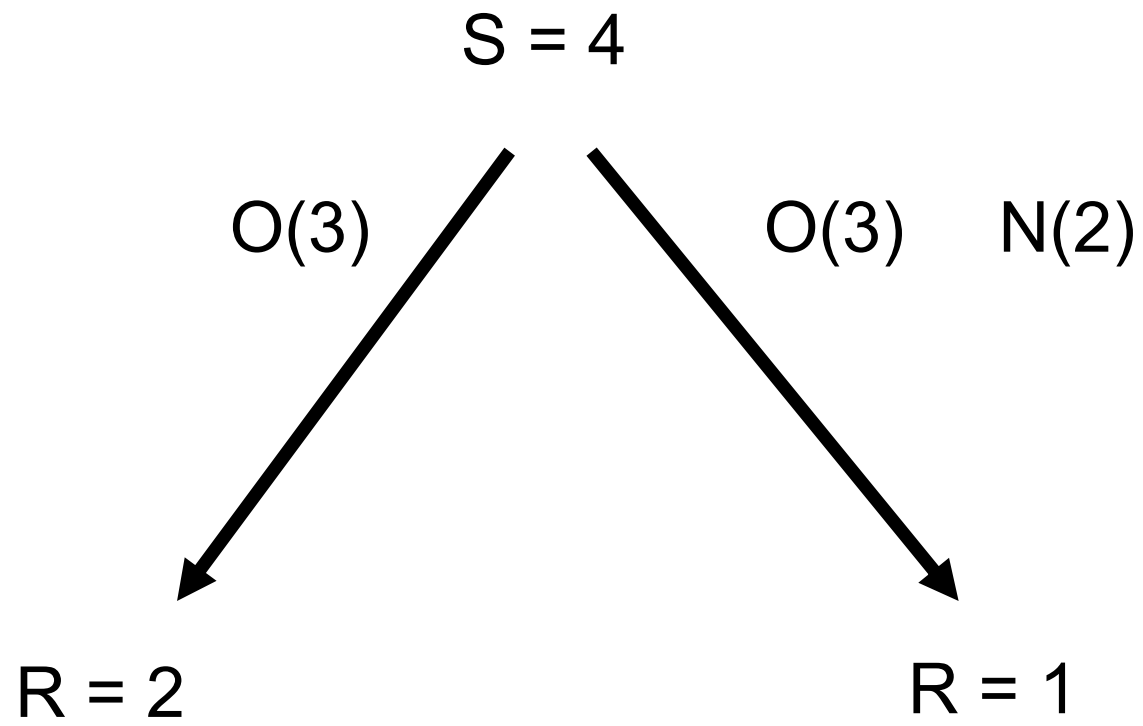


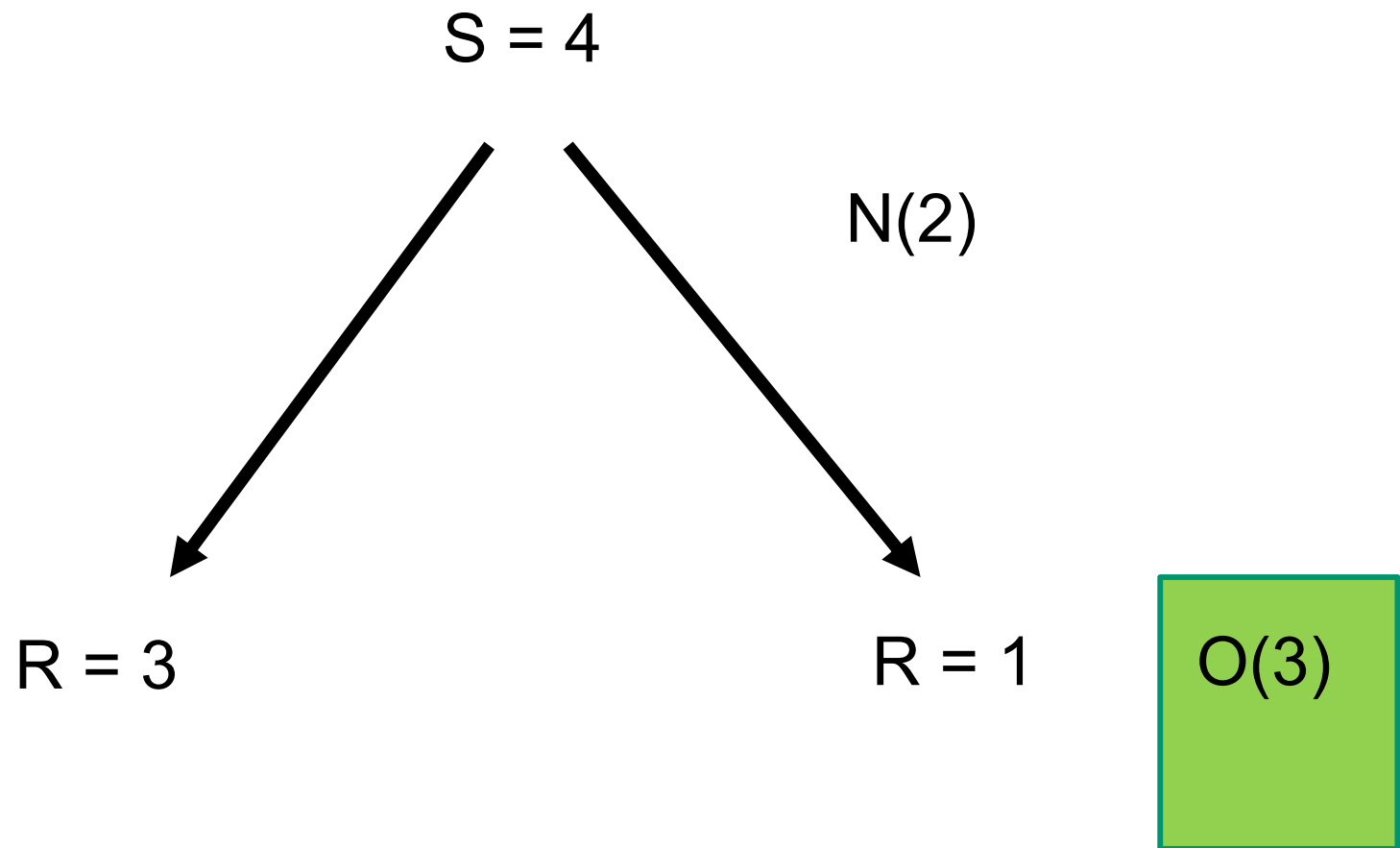
$S = 2$

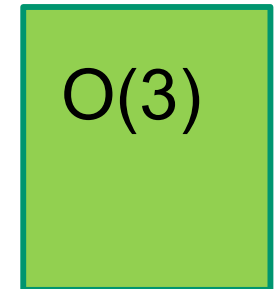
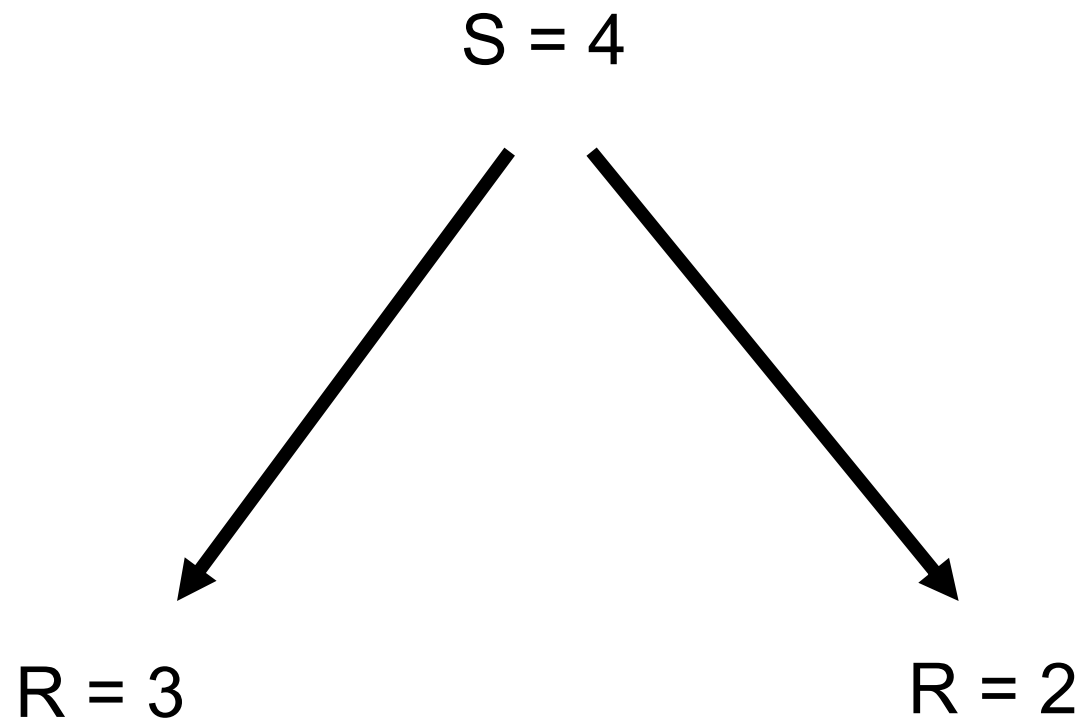




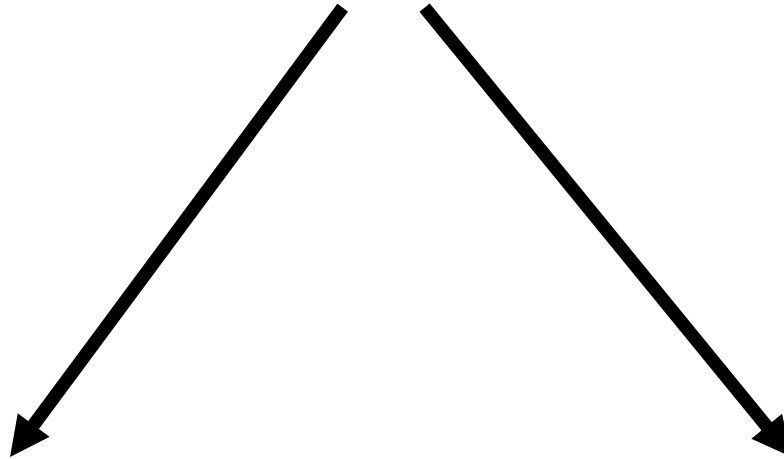








$S = 4$

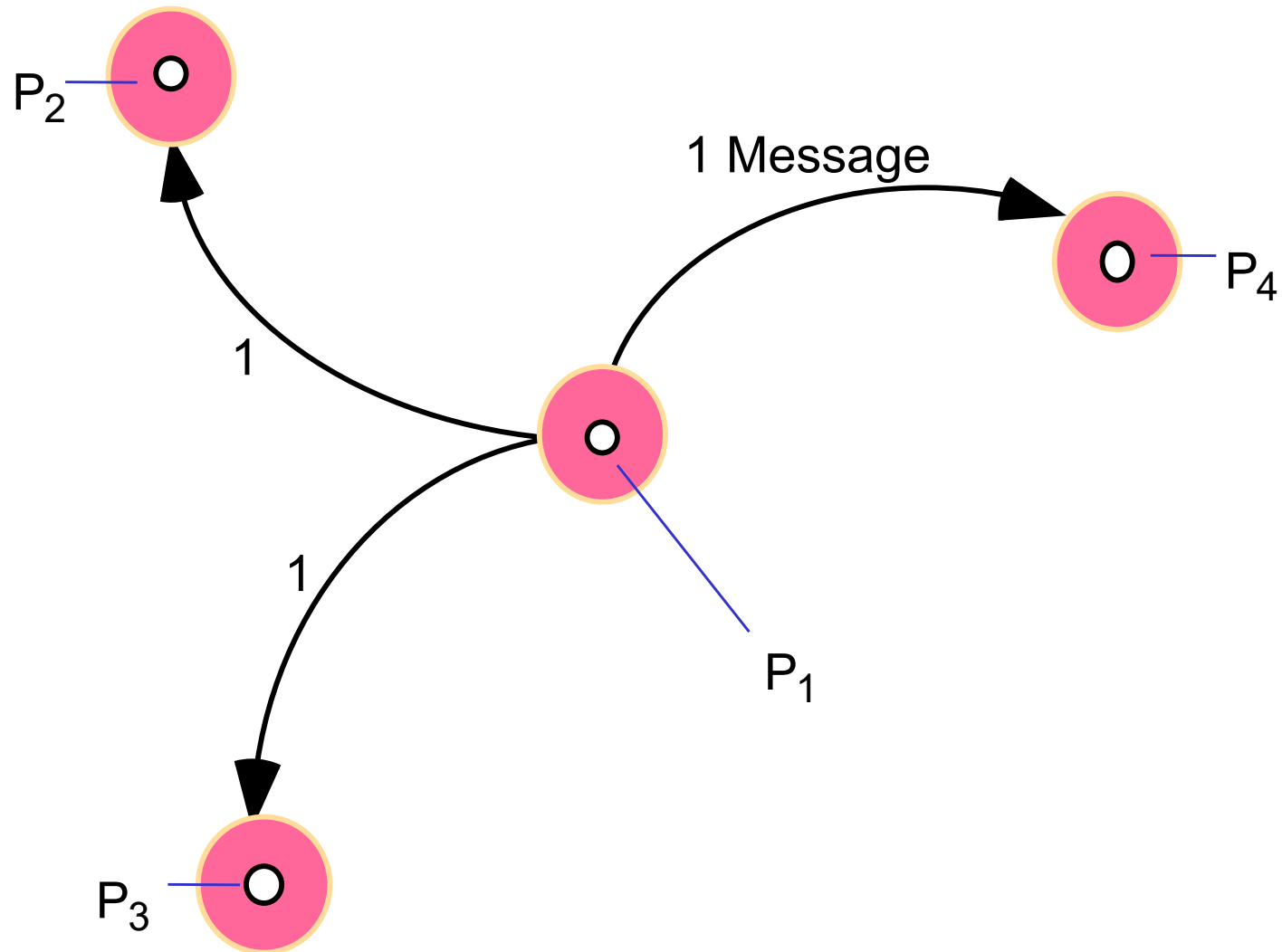


$R = 3$

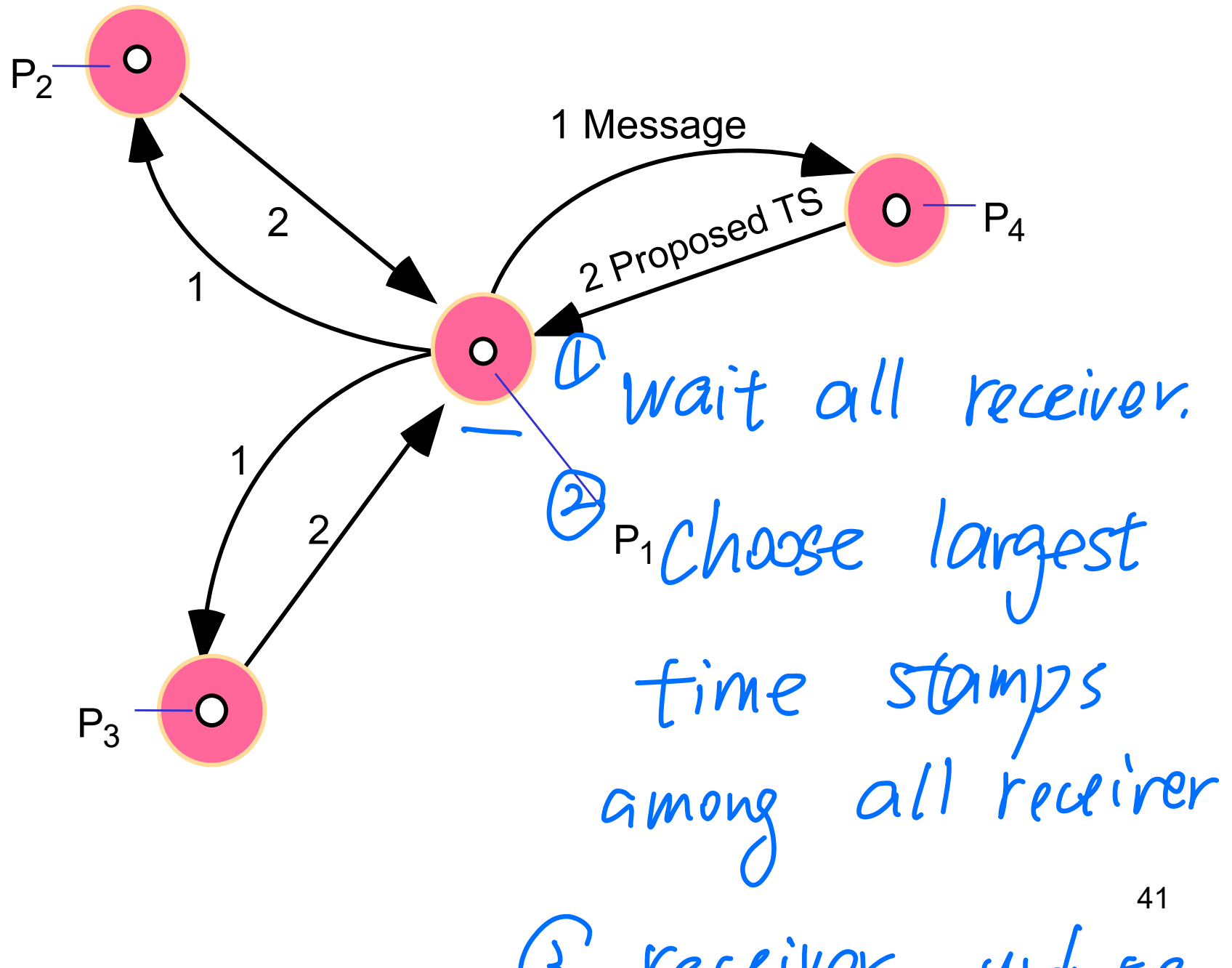
$R = 3$

begin

Implementing total ordering

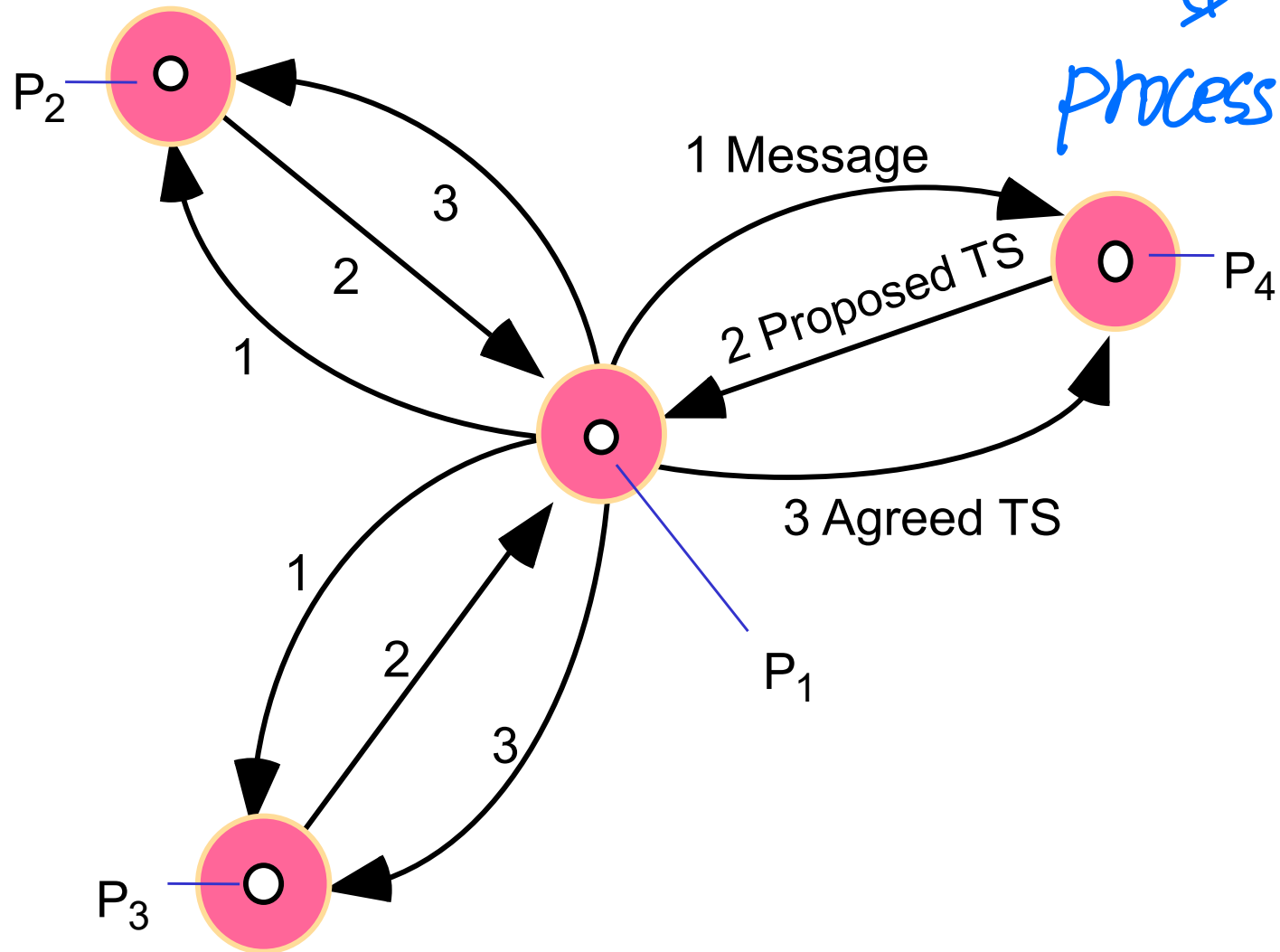


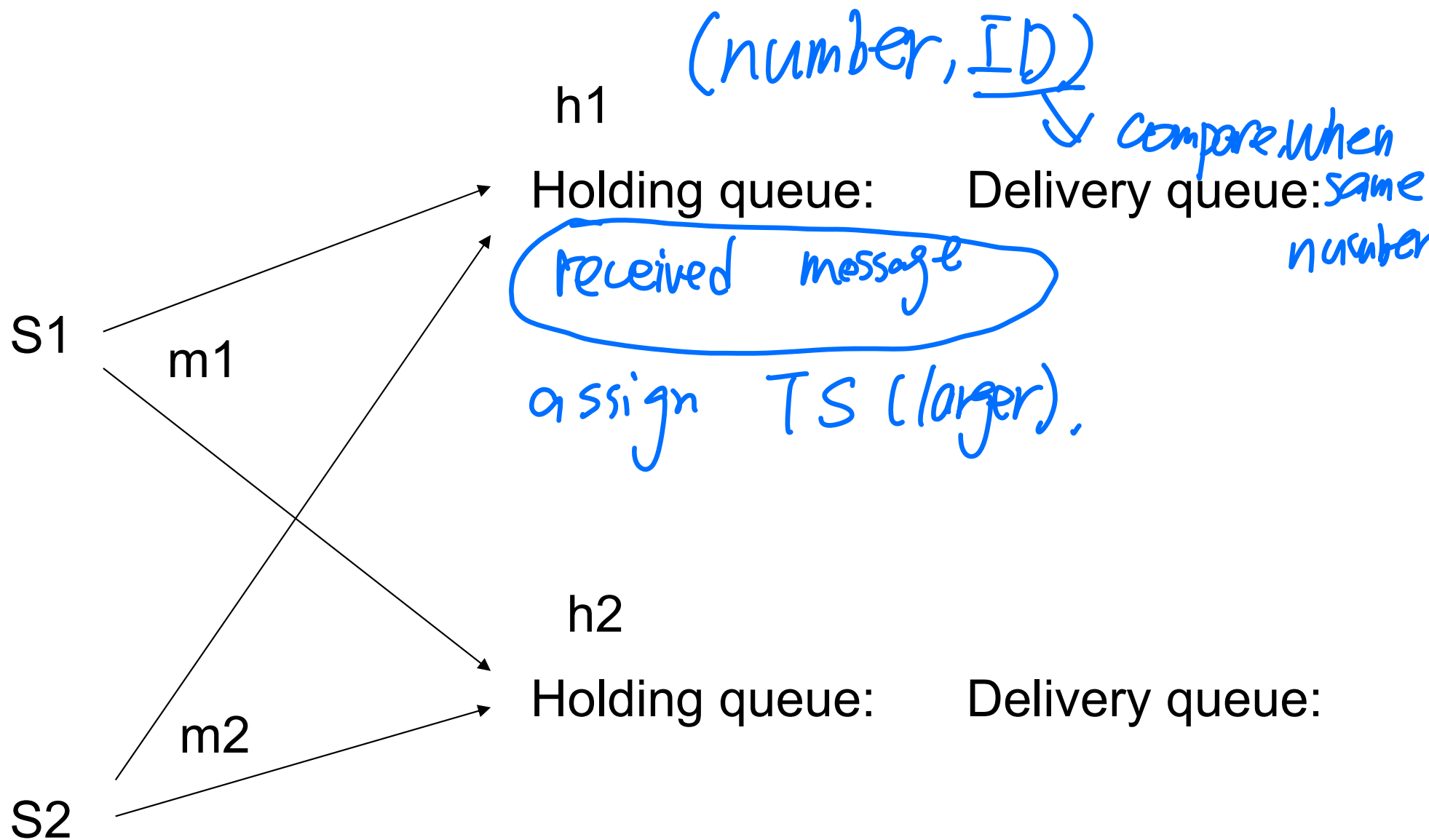
Implementing total ordering

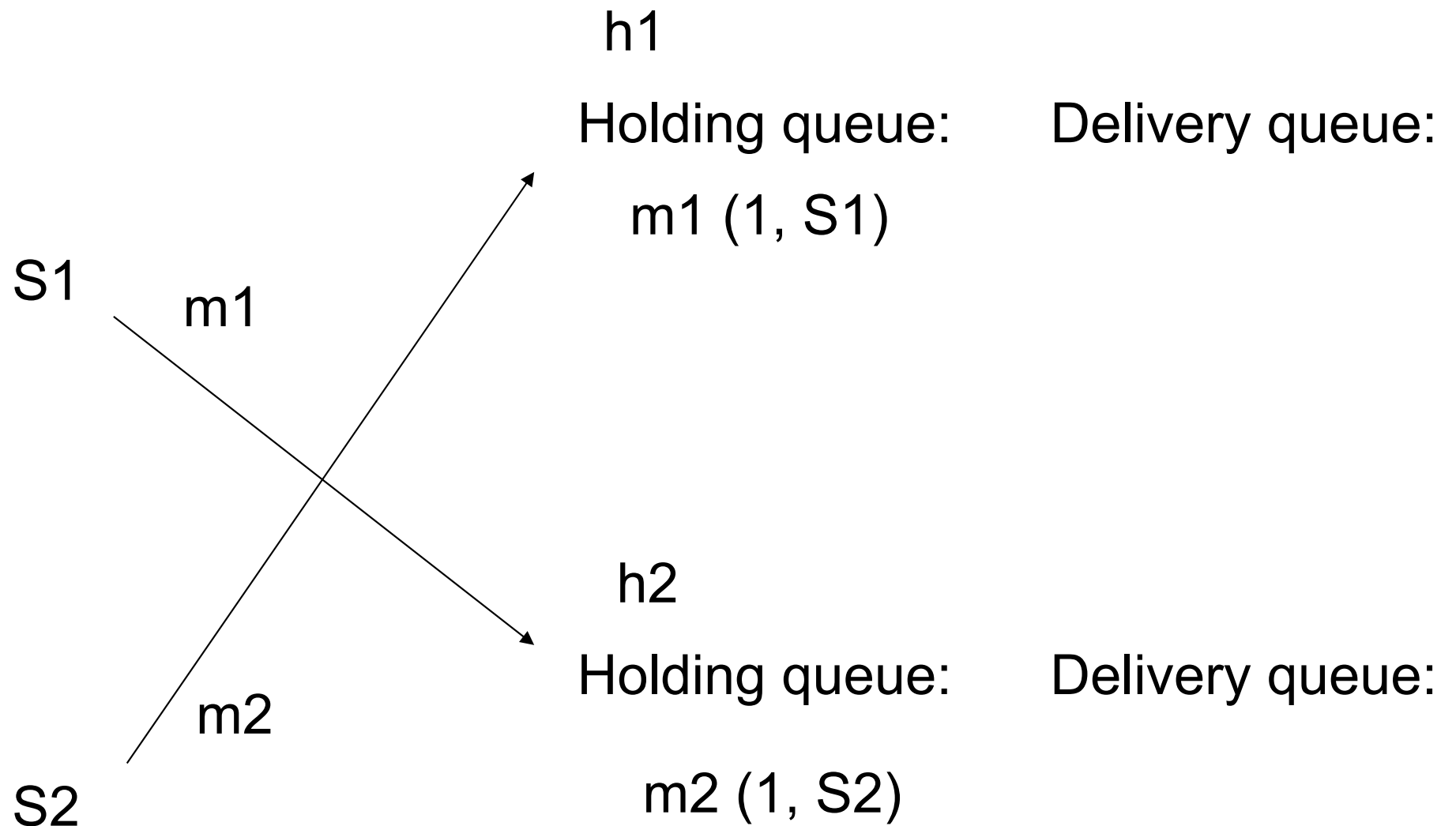


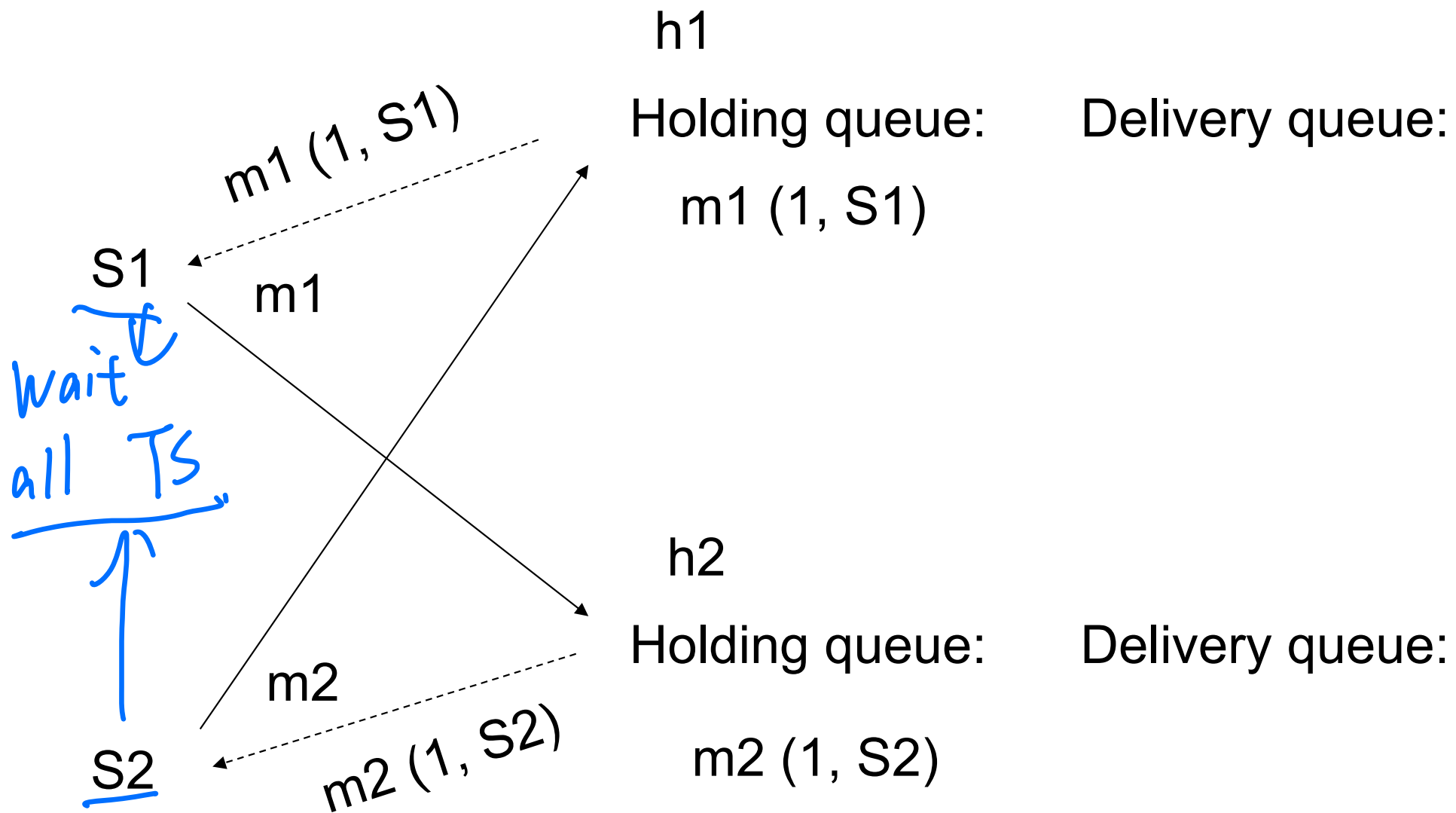
Implementing total ordering

Receiver
message
time stamp
update
↓
process message









h1

Holding queue:

Delivery queue:

m1 (1, S1)

m2 (2, S2)

S1

m1 (1, S1)

h2

Holding queue:

Delivery queue:

m2 (1, S2)

m1 (2, S1)

S2

m2 (1, S2)

h1

Holding queue:

m1 (1, S1)

m2 (2, S2)

Delivery queue:

h2

Holding queue:

m2 (1, S2)

m1 (2, S1)

Delivery queue:

S1

m1 (1, S1)

S2

m2 (1, S2)

m2 (2, S2)

m1 (2, S1)

h1

Holding queue:

Delivery queue:

m1 (1, S1)

m2 (2, S2)

S1

m1 (1, S1)

m1 (2, S1)

choose larger

h2

Holding queue:

Delivery queue:

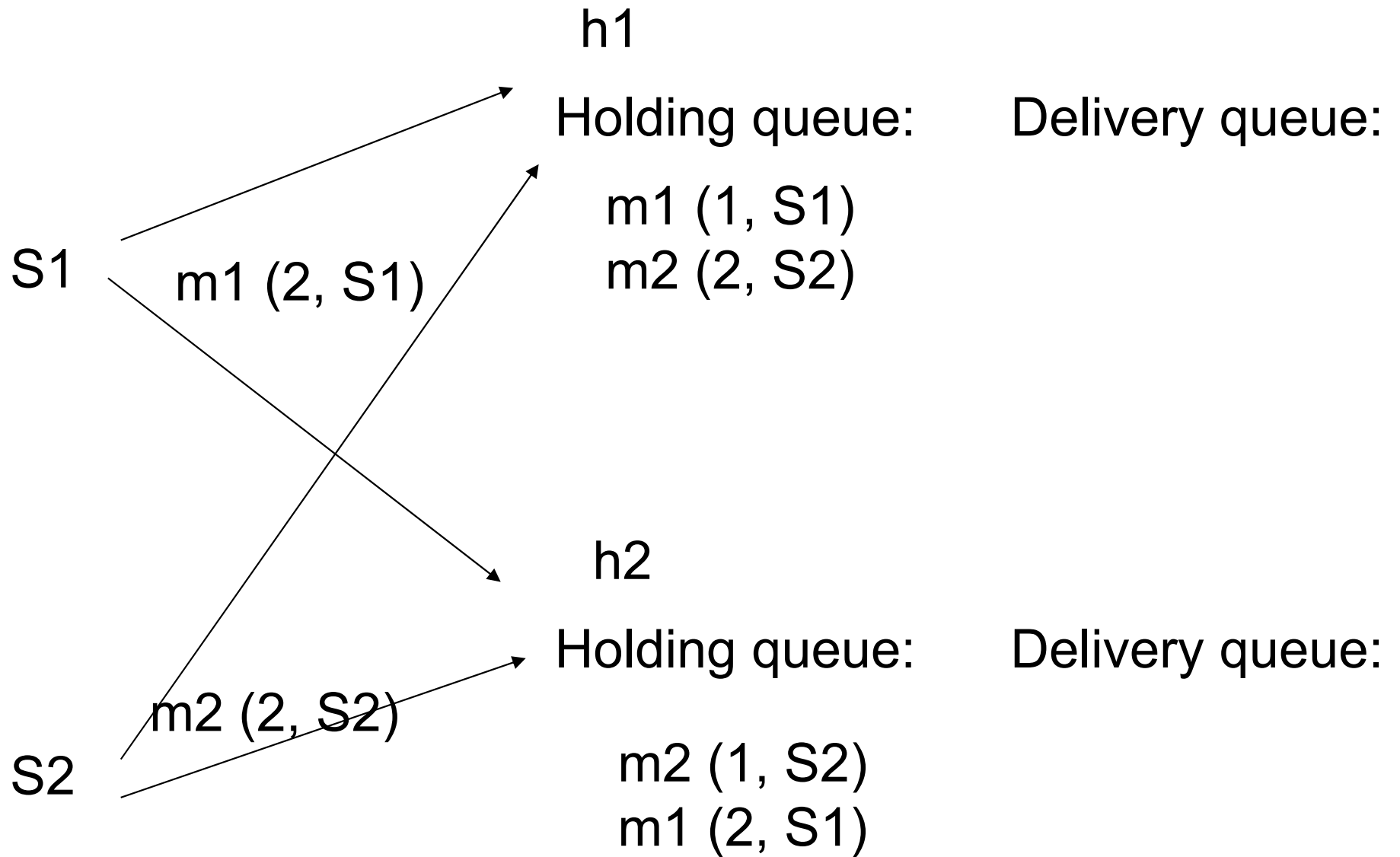
m2 (1, S2)

m1 (2, S1)

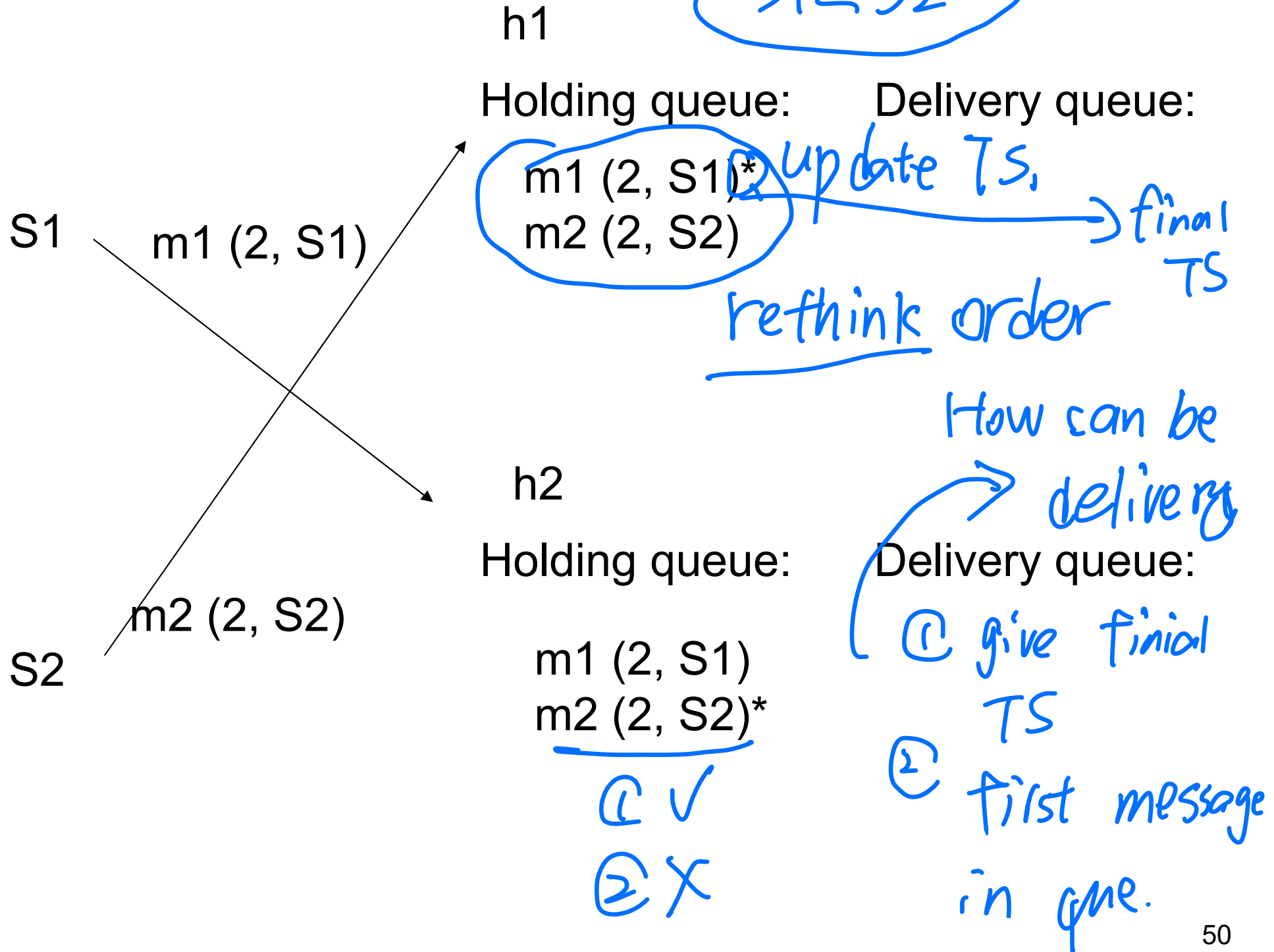
S2

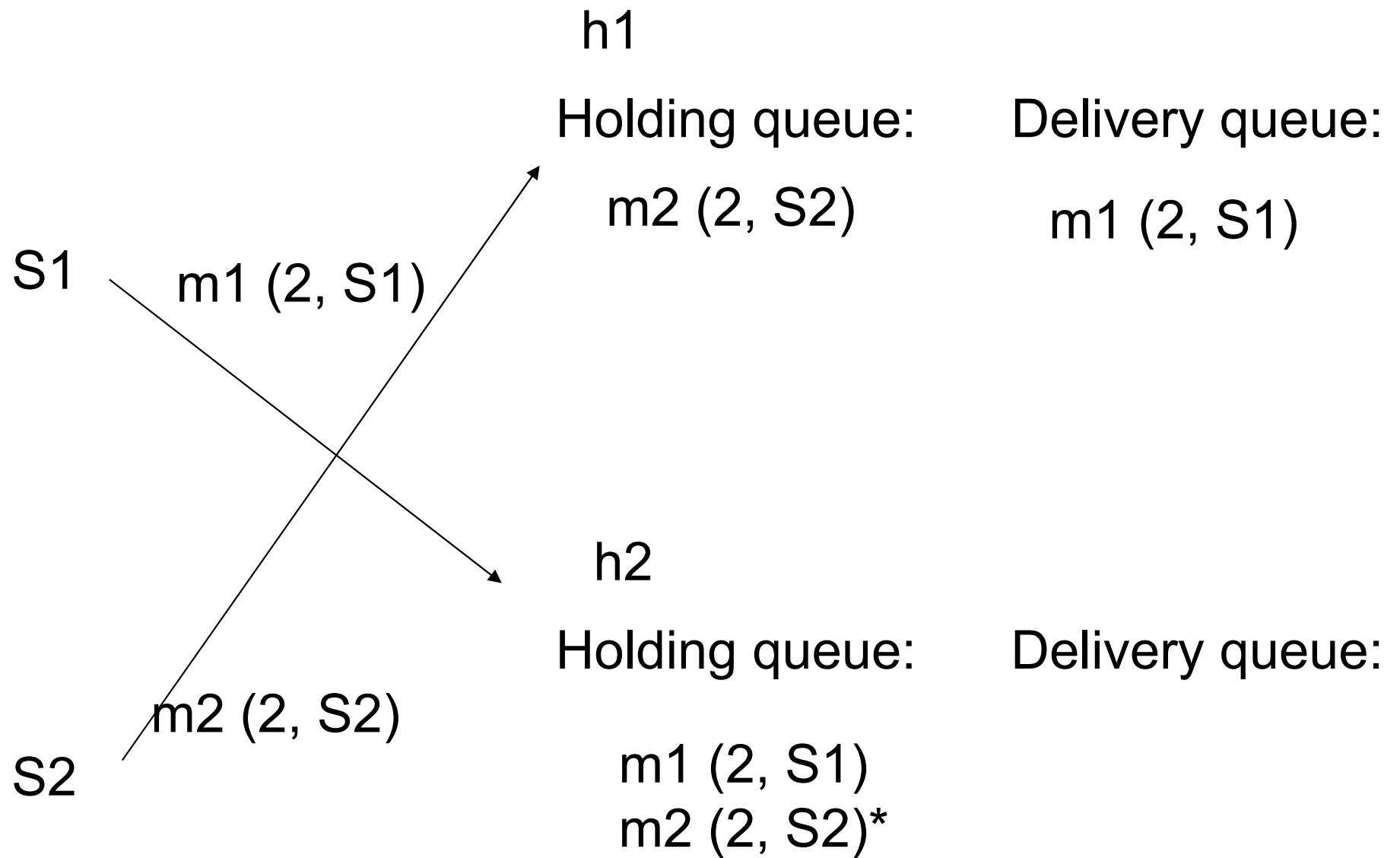
m2 (1, S2)

m2 (2, S2)



$$S_1 < S_2$$





h1

Holding queue:

m2 (2, S2)*

Delivery queue:

m1 (2, S1)

S1

h2

Holding queue:

m1 (2, S1)*

m2 (2, S2)*

Delivery queue:

S2

h1

Holding queue:

Delivery queue:

S1

m1 (2, S1)

m2 (2, S2)

h2

Holding queue:

Delivery queue:

S2

m1 (2, S1)

m2 (2, S2)

- ❑ The sender transmits msg to its destinations.
- ❑ Each recipient adds the message to the priority queue associated with label, tagging it as undeliverable. It assigns this message a priority larger than the priority of any message that was placed in the queue, with the process ID of the sender as a suffix. It then informs the sender of the priority that it assigned to the message.
- ❑ The sender collects responses from recipients that remain operational. It then computes the maximum value of all the priorities it received, and sends this value back to the recipients.
- ❑ The recipients change the priority of the message to the value they receive from the sender, tag the message as deliverable, and re-sort their priority queues. They then transfer messages from the priority queue to the delivery queue in order of increasing priority, until the priority queue becomes empty or the message with the lowest priority is undeliverable.

Implementing causal ordering

- Assumption: the communication between two processes has the **FIFO property**
 - who send first
 - who receive first
- Each process maintains **a set of variables**.
 - A **variable** records the multicast messages received from a process in the multicast group
 - A **vector** represents the collection of these variables
 - How many message has been process
- When a process multicasts a message, it **attaches** the values of these variables to the message
 - The values imply the number of multicast messages that precede the one being multicast
 - Attach the vector
- When a process receives a multicast message, the process only **delivers** the message after the preceding multicast messages have been received

$$\begin{aligned} V_1[1] &= 0 \\ V_1[2] &= 0 \\ V_1[3] &= 0 \end{aligned}$$



P_1

$$\begin{aligned} V_2[1] &= 0 \\ V_2[2] &= 0 \\ V_2[3] &= 0 \end{aligned}$$



P_2

$$\begin{aligned} V_3[1] &= 0 \\ V_3[2] &= 0 \\ V_3[3] &= 0 \end{aligned}$$



P_3

第i变量

$V_1[1]=0$
 $V_1[2]=0$
 $V_1[3]=0$

$V_2[1]=0$
 $V_2[2]=0$
 $V_2[3]=0$

$V_3[1]=0$
 $V_3[2]=0$
 $V_3[3]=0$

send out message

M1

$V_1[1]=1$

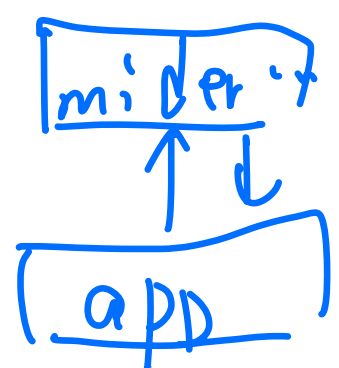
$\begin{cases} V_1[2]=0 \\ V_1[3]=0 \end{cases}$

M1

M1

M1

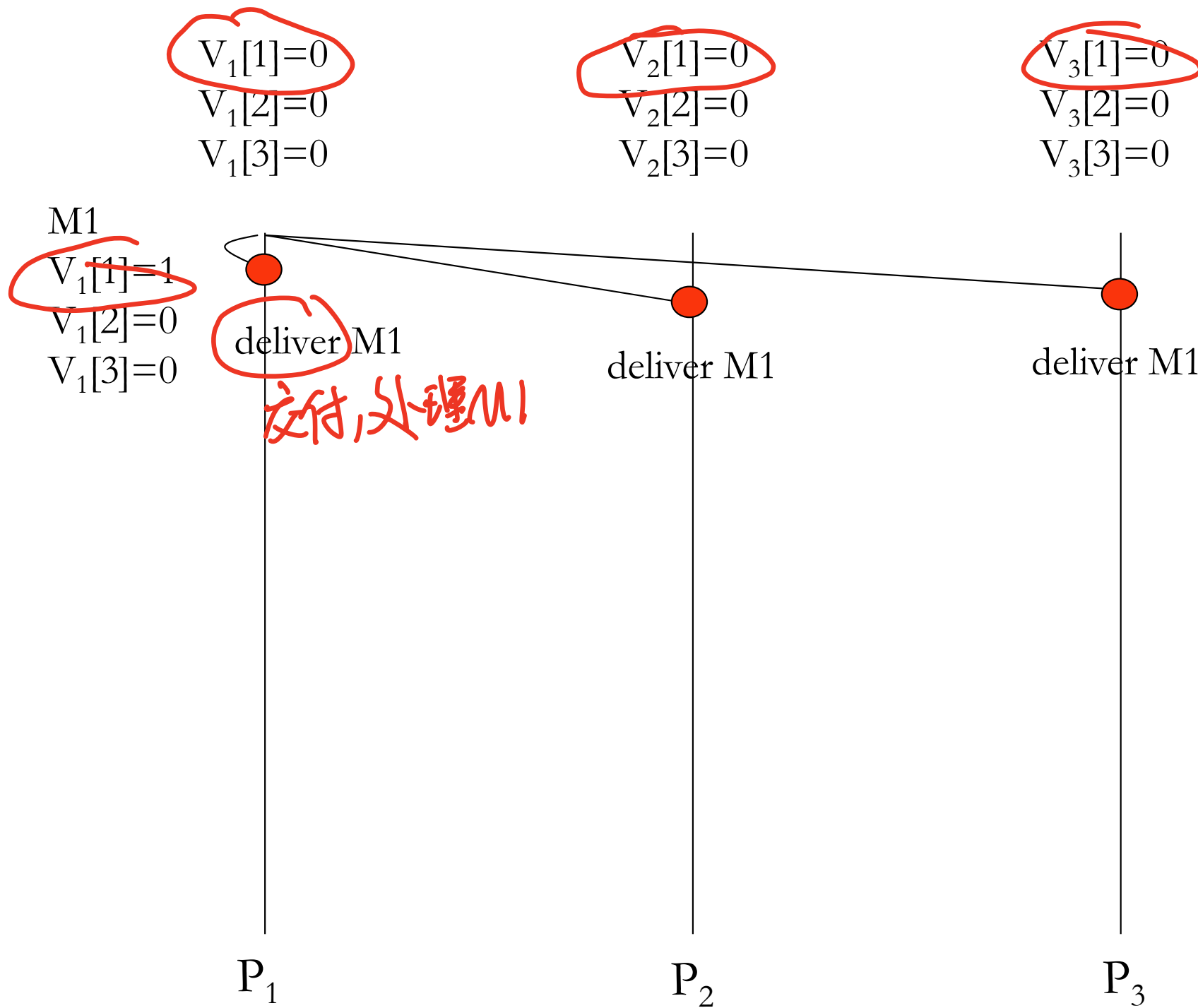
before sent
out not receive
other message



P_1

P_2

P_3

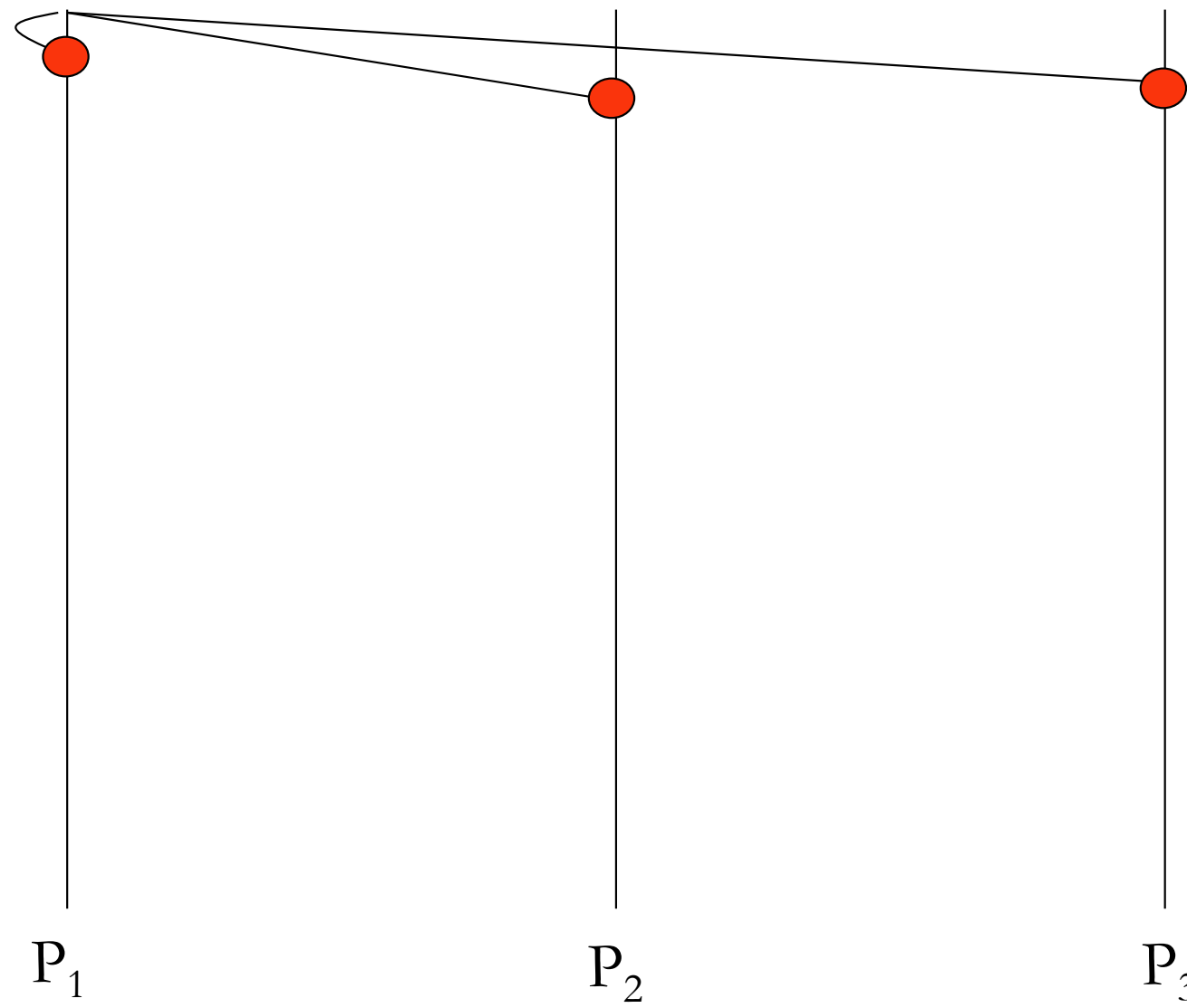


update TS

$V_1[1]=1$
 $V_1[2]=0$
 $V_1[3]=0$

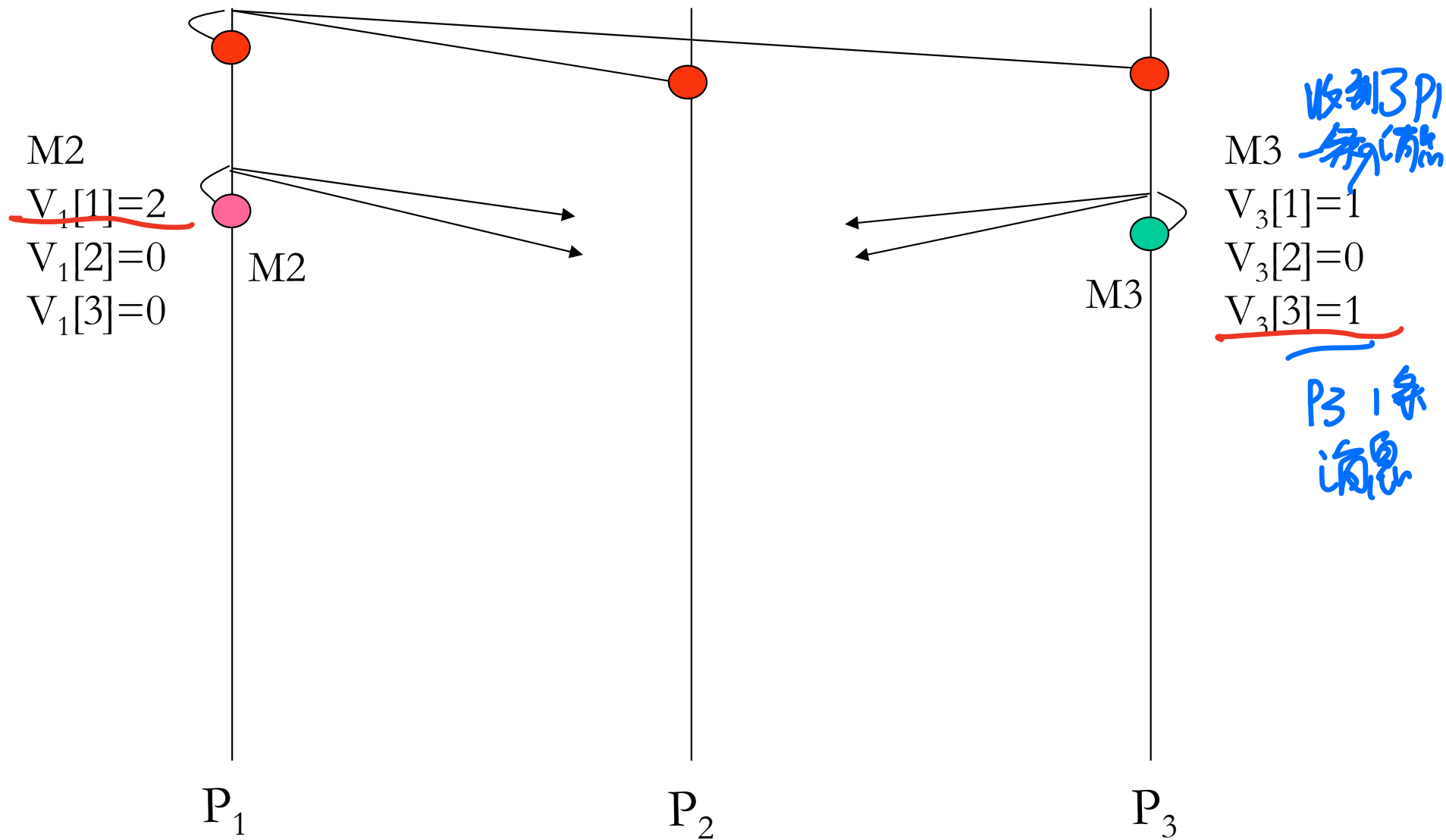
$V_2[1]=1$
 $V_2[2]=0$
 $V_2[3]=0$

$V_3[1]=1$
 $V_3[2]=0$
 $V_3[3]=0$

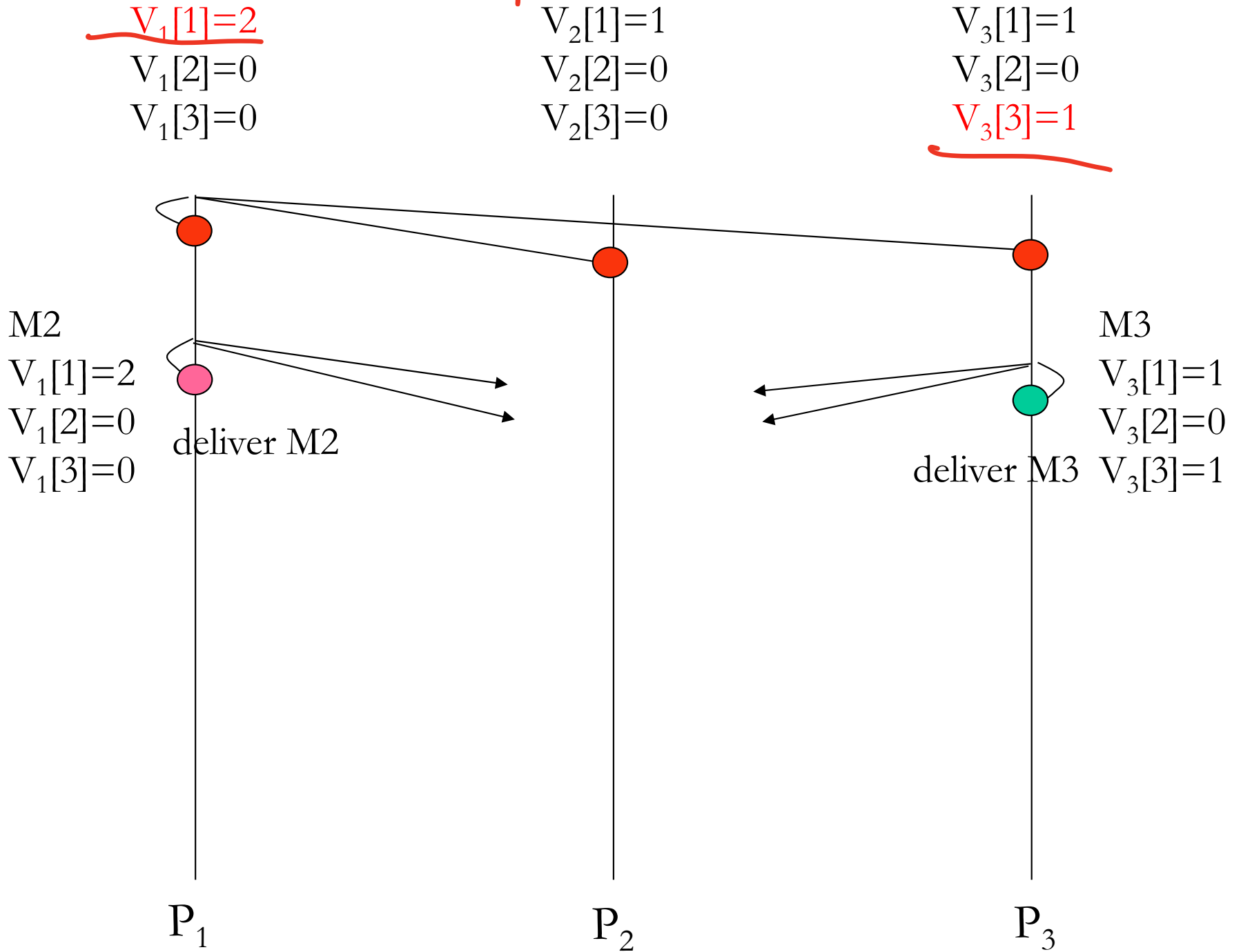


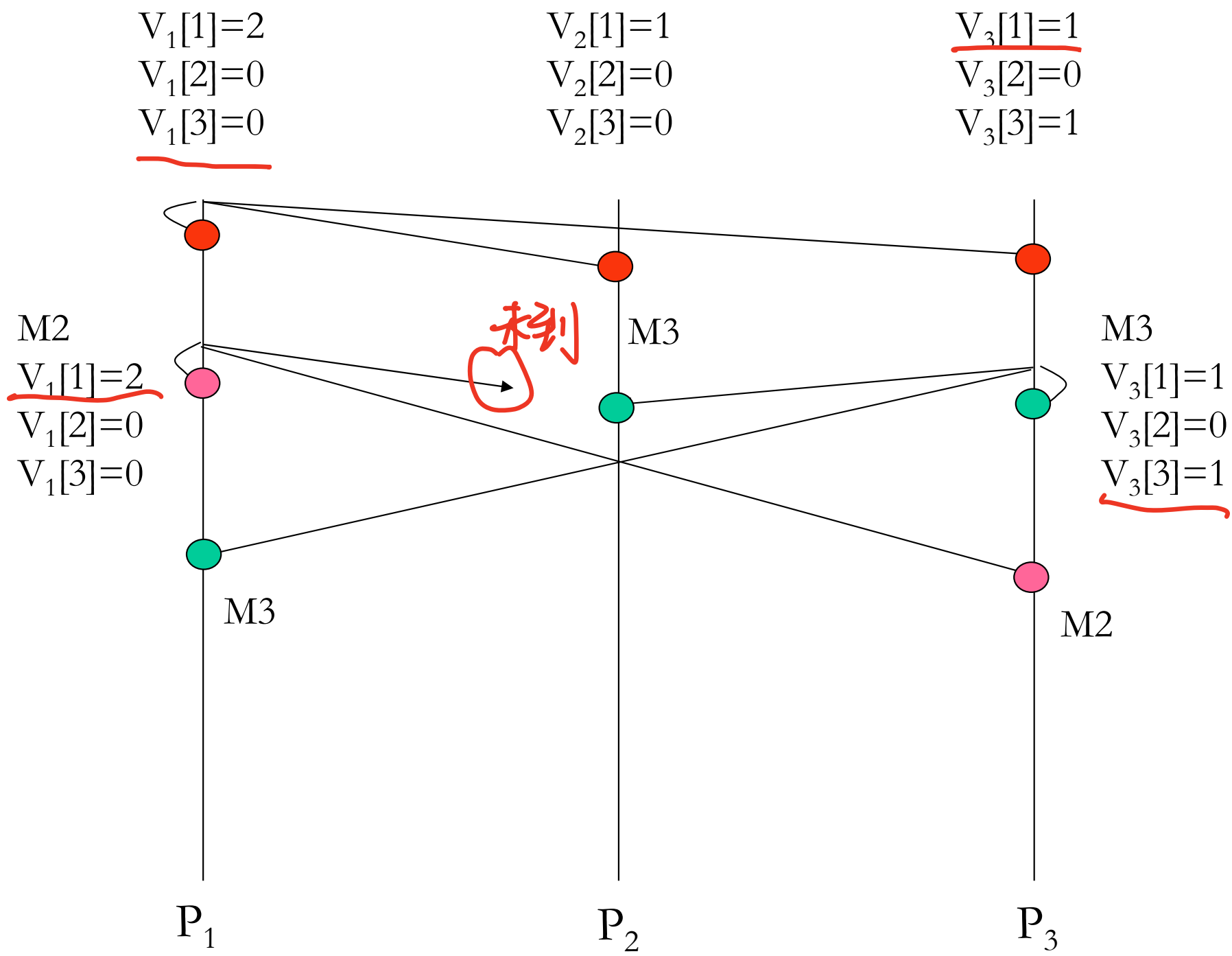
$$\begin{aligned} V_1[1] &= 1 \\ V_1[2] &= 0 \\ V_1[3] &= 0 \end{aligned}$$

$$\begin{aligned} V_2[1] &= 1 \\ V_2[2] &= 0 \\ V_2[3] &= 0 \end{aligned}$$

$$\begin{aligned} V_3[1] &= 1 \\ V_3[2] &= 0 \\ V_3[3] &= 0 \end{aligned}$$


up date TS





交付

$V_1[1]=2$
 $V_1[2]=0$
 $V_1[3]=0$

收到消息

$V_2[1]=1$
 $V_2[2]=0$
 $V_2[3]=0$

收到消息
↓
交付

$V_3[1]=1$
 $V_3[2]=0$
 $V_3[3]=1$

收到1条
↓
交付

发出消息

M2
 $V_1[1]=2$
 $V_1[2]=0$
 $V_1[3]=0$

deliver M3

M3
 $V_3[1]=1$
 $V_3[2]=0$
 $V_3[3]=1$

发出消息

deliver M3

deliver M2

P_1

P_2

P_3

① 发出消息
② 交付后更新 → 进行时间

戏

$V_1[1]=2$
 $V_1[2]=0$
 $V_1[3]=1$

$V_2[1]=1$
 $V_2[2]=0$
 $V_2[3]=1$

$V_3[1]=2$
 $V_3[2]=0$
 $V_3[3]=1$

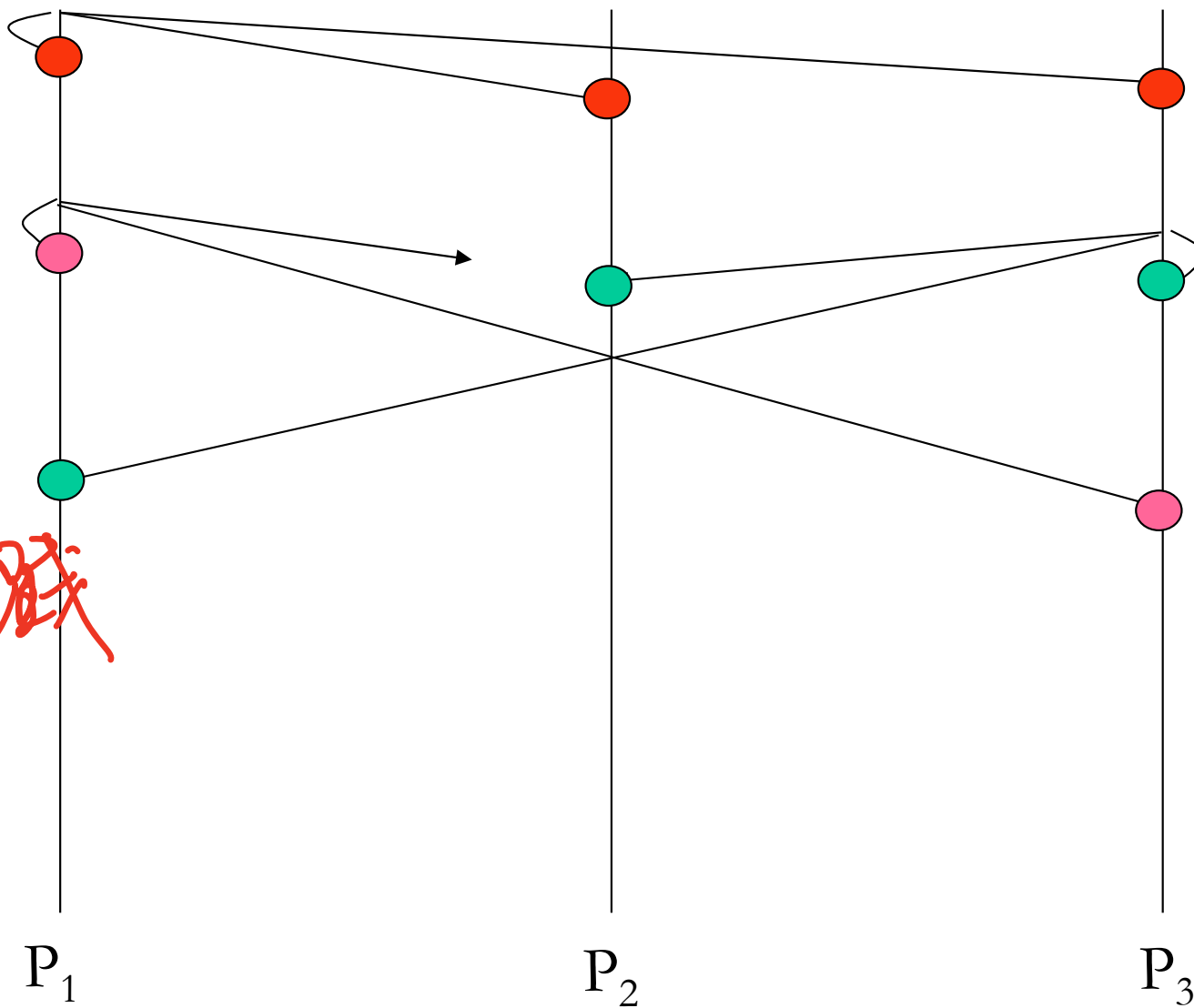
M2

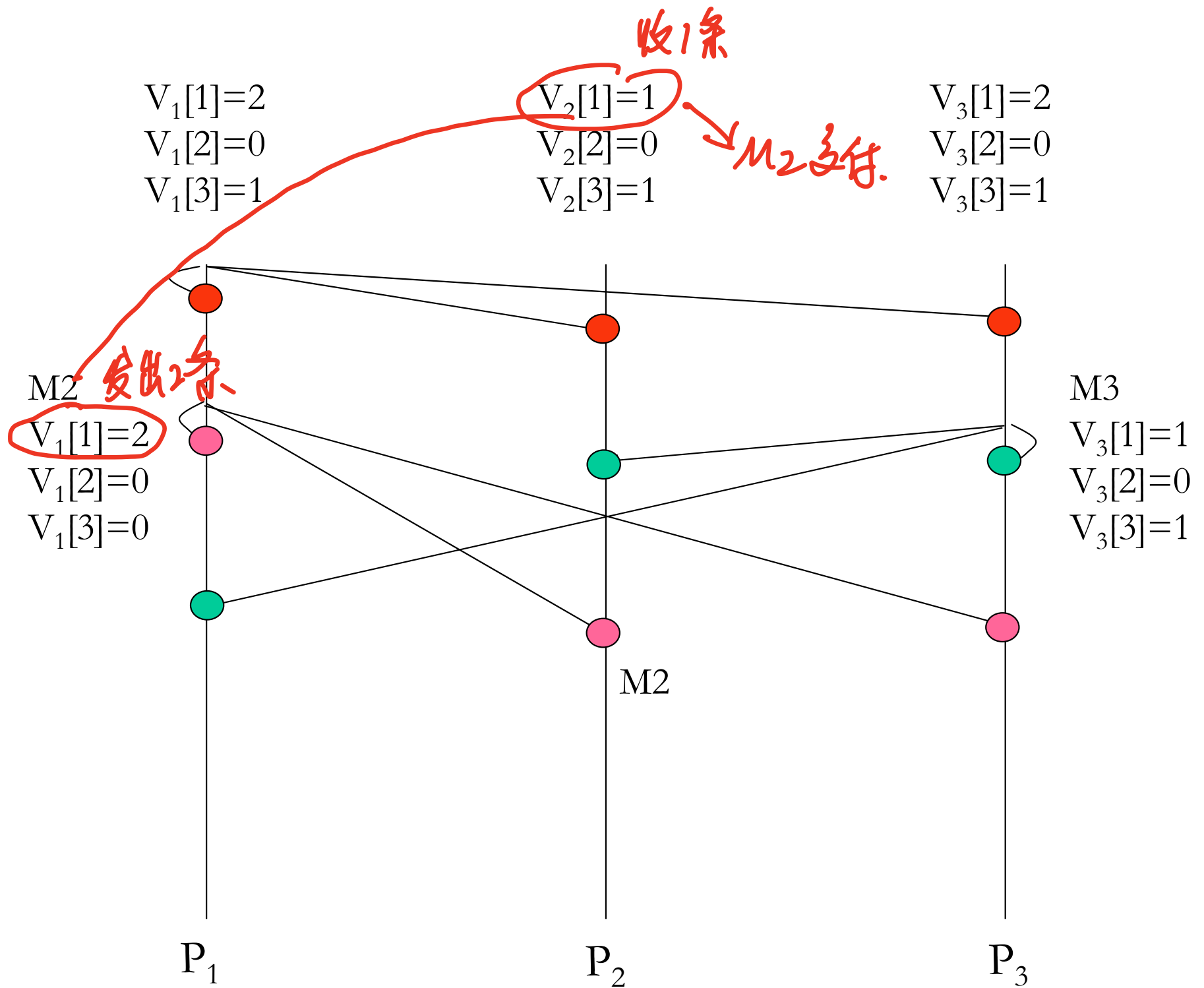
$V_1[1]=2$
 $V_1[2]=0$
 $V_1[3]=0$

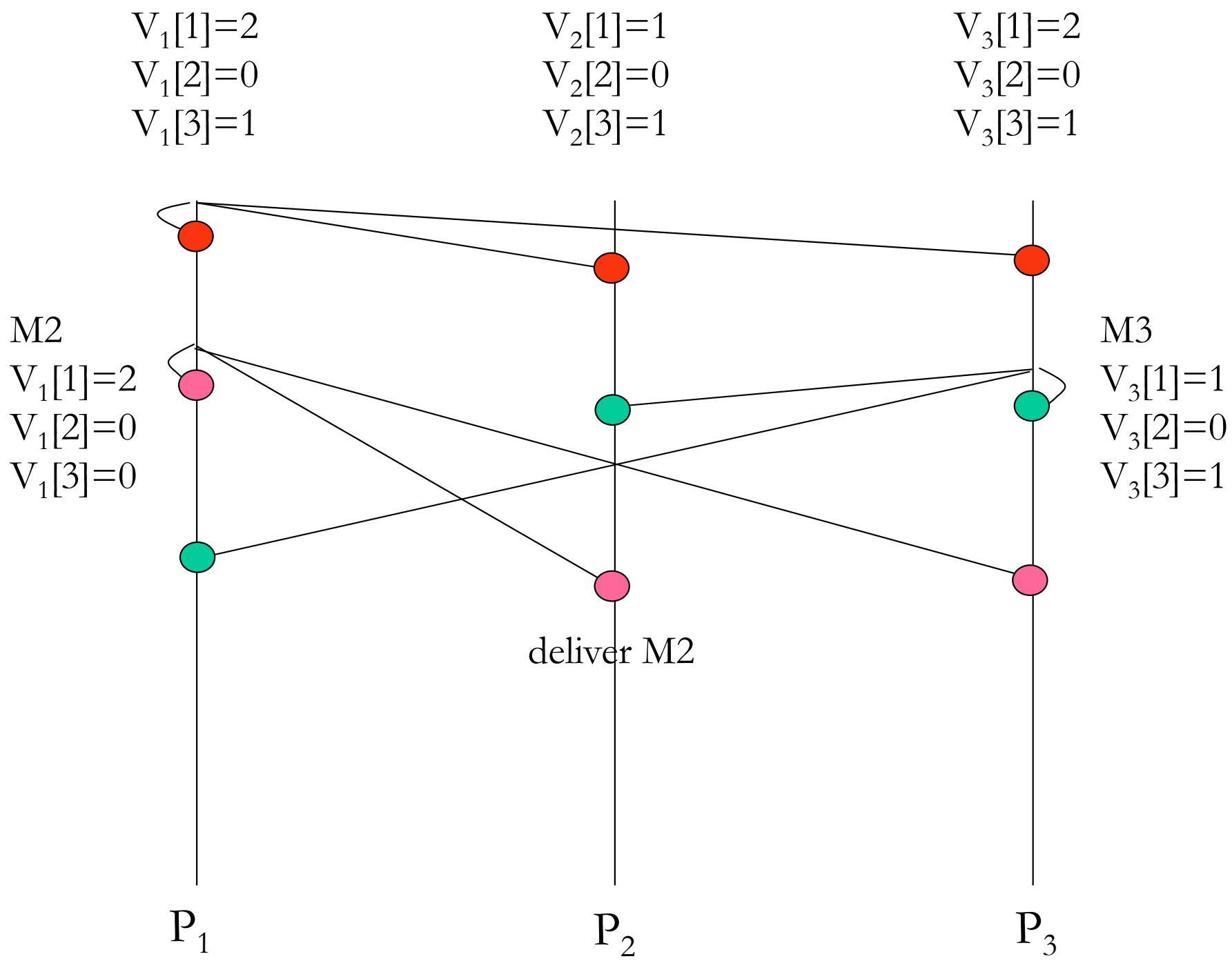
M3

$V_3[1]=1$
 $V_3[2]=0$
 $V_3[3]=1$

更新时间戳



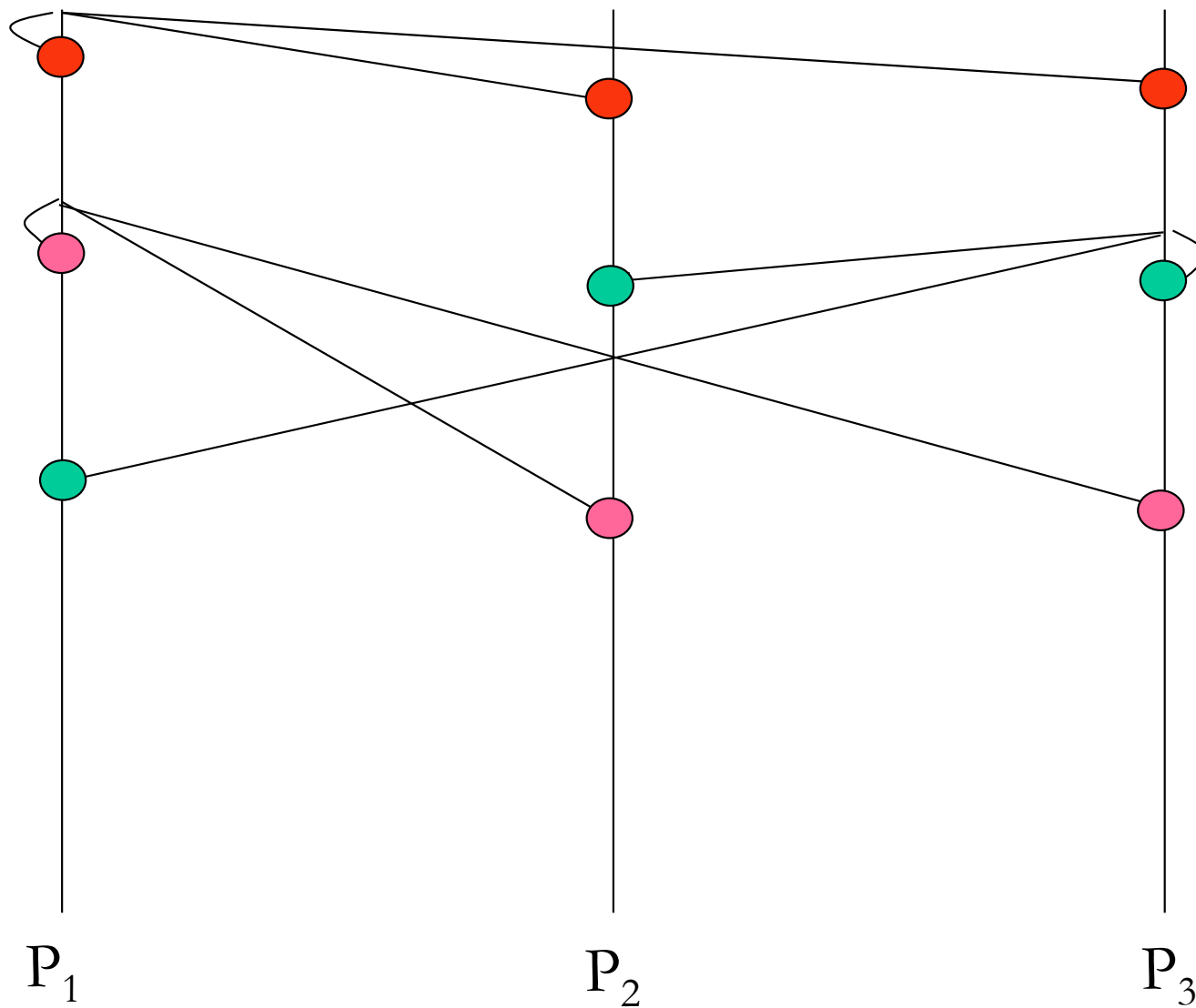




$$\begin{aligned} V_1[1] &= 2 \\ V_1[2] &= 0 \\ V_1[3] &= 1 \end{aligned}$$

$$\begin{aligned} V_2[1] &= 2 \\ V_2[2] &= 0 \\ V_2[3] &= 1 \end{aligned}$$

$$\begin{aligned} V_3[1] &= 2 \\ V_3[2] &= 0 \\ V_3[3] &= 1 \end{aligned}$$



- Let $V_i[j]$ represents the message received by process i from process j
- Initially, on each process $V_i[j] = 0$ ($1 \leq j \leq N$)
- When process i multicasts a message $V_i[i]$ is incremented
- Assume process i receives a message M from process j . The following conditions must be satisfied before M is delivered:

- $V_j[j] = V_i[j] + 1$

- Ensure FIFO order between i and j

- $V_j[k] \leq V_i[k]$ ($k \neq j$)

- Ensure that multicast messages from other processes that precedes M are delivered first.

- 一个进程发出的消息 a, b, c

a 收到后才能收 b, c

b 收到后才能收 c

若 a 后 c 到, c 进等待

队列等 b

到交付.

Reliable multicast

- ❑ For some applications, a multicast message is required to be delivered either to **all** the processes or to **none** at all.
 - A command sent to a group of players in an on-line game
- ❑ Reliable multicast means that every process in the process group receives all multicasts
- ❑ Reliability is **orthogonal** to ordering
 - Can implement Reliable-FIFO, or Reliable-Causal, or Reliable-Total

- ❑ Since processes might **crash** during its execution, ensuring reliable multicast is a non-trivial problem.
 - A process might have delivered a multicast messages to **some processes** and crashed before it is able to deliver the message to the **other processes** in the multicast group
- ❑ We want all the **nonfaulty** processes receive the same set of multicast messages
- ❑ The faulty processes will not participate in the computation. So, they are not a concern.

A naive algorithm for achieving reliable multicast

- ❑ The sender uses TCP to send multicast message to each receiver.
- ❑ To ensure a message is received by all processes in a group, each receiver sends a received multicast message to all the other processes in the group except the sender of the multicast message.
 - A receiver would not re-multicast the same message more than once.
- ❑ What's the message complexity?

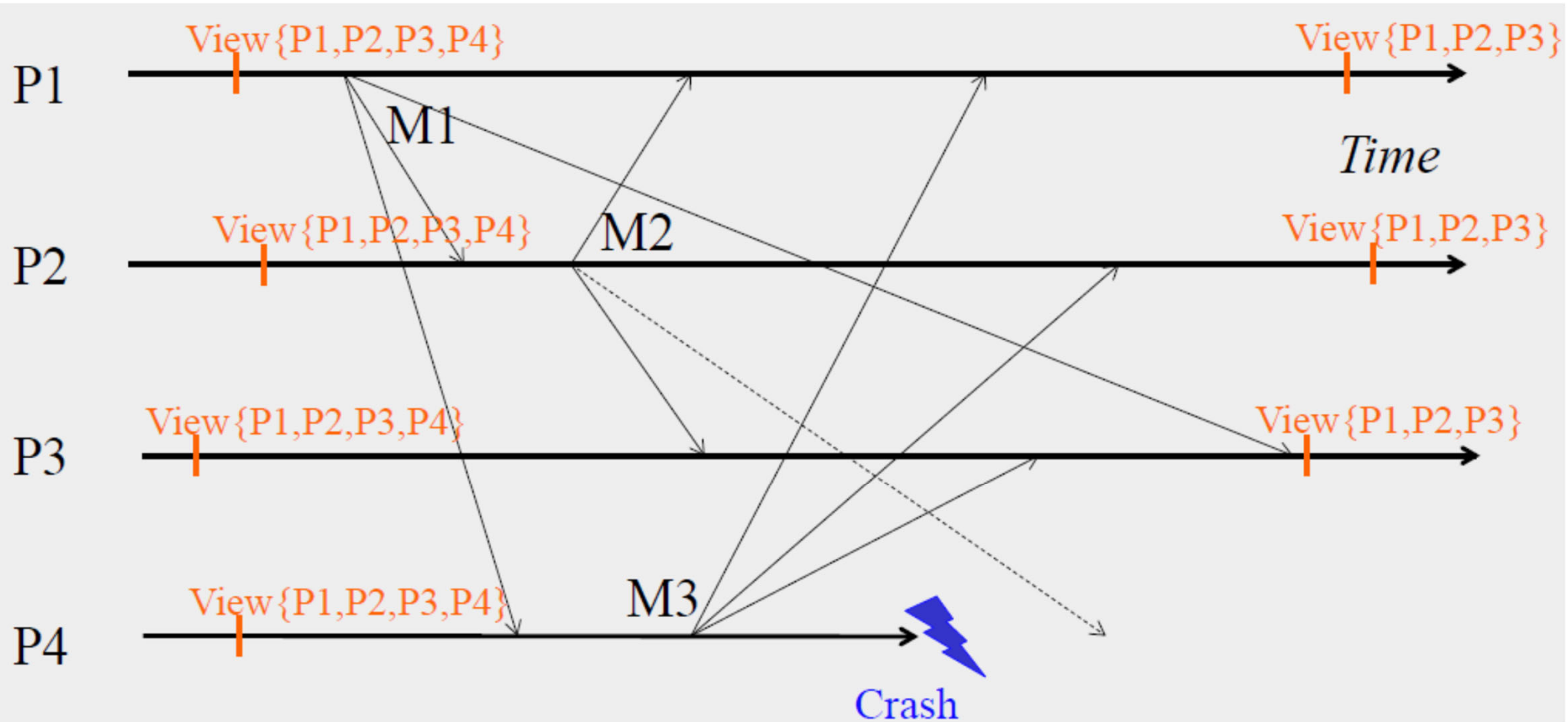
Virtual Synchrony

- ❑ Virtual synchrony is a model for **managing** group membership and group communications.
- ❑ Processes might **join or leave** a multicast group (due to failure or planned leave).
- ❑ A **view** of a group refers to the membership of a multicast group at a given point of time.
- ❑ When processes join or leave a group, the view of the group **changes**.
- ❑ The processes in a group should have the **same group view** when a message is multicast.
 - All the processes agree who should receive the message.
- ❑ When a process join or leave a group, a **view change message** should be broadcast to all the processes.
 - The processes should change their view together.

- Virtual synchrony guarantees that all **view changes** are delivered in the **same order** at all non-faulty processes
 - If process P1 receives view changes, say {P1}, {P1, P2, P3}, {P1, P2}, {P1, P2, P4} then non-faulty process receives the *same sequence* of view changes (after it joins the group)
 - E.g., P2 receives views {P1, P2, P3}, {P1, P2}, {P1, P2, P4}

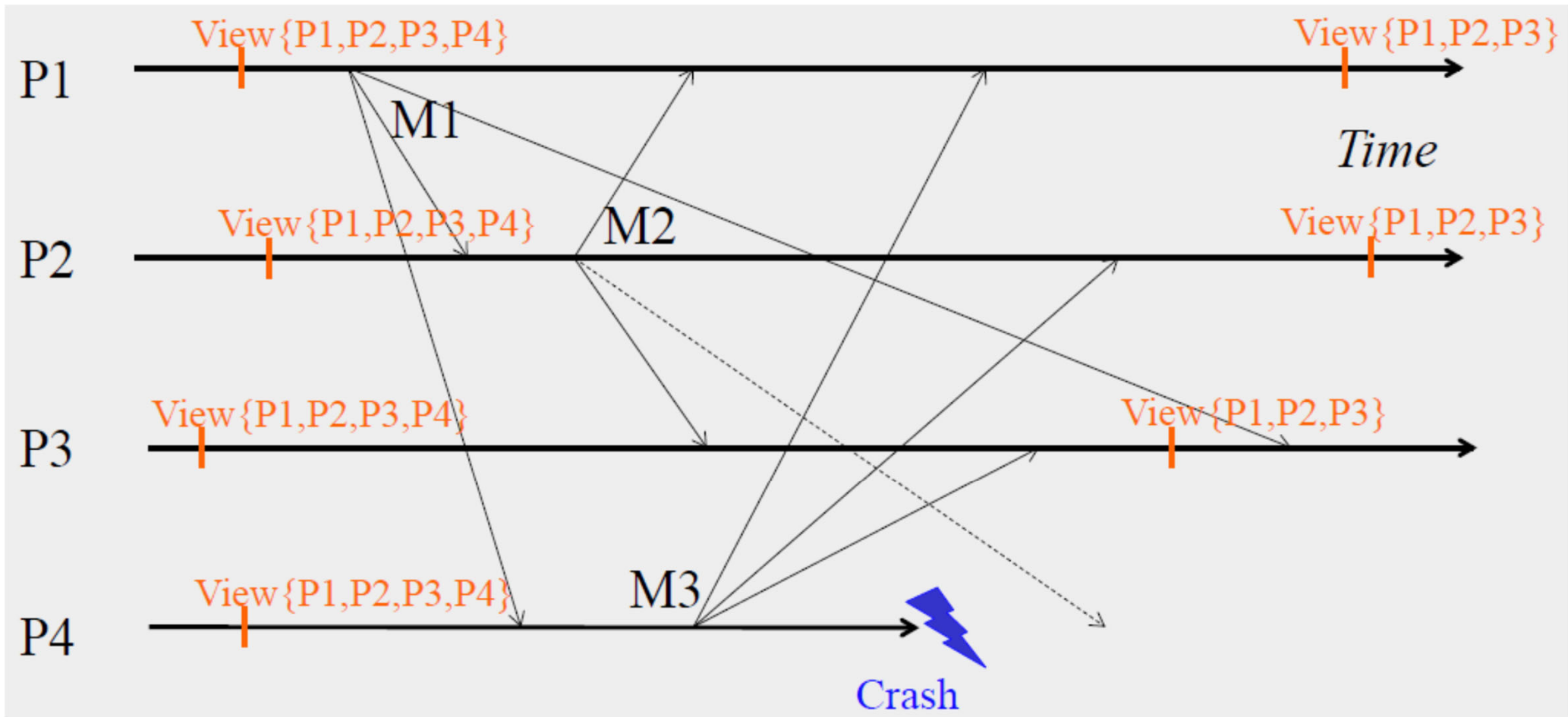
- A multicast message M is said to be “**delivered in a view V at process P** ” if P receives view V and delivers M before P receives the next view
- Virtual synchrony **ensures** that
 - The set of multicasts delivered **in a given view** is the **same** at all non-faulty processes that were in that view
 - The sender of the multicast message also **belongs** to that view
 - If the sender of a message **crashes** during the multicast, the message may either be delivered to all nonfaulty processes or ignored by each of them.
- A multicast with this property is called **virtually synchronous**.
- Systems that implemented it have been used in many mission-critical information systems, e.g. NYSE, Air Traffic Control System.

examples

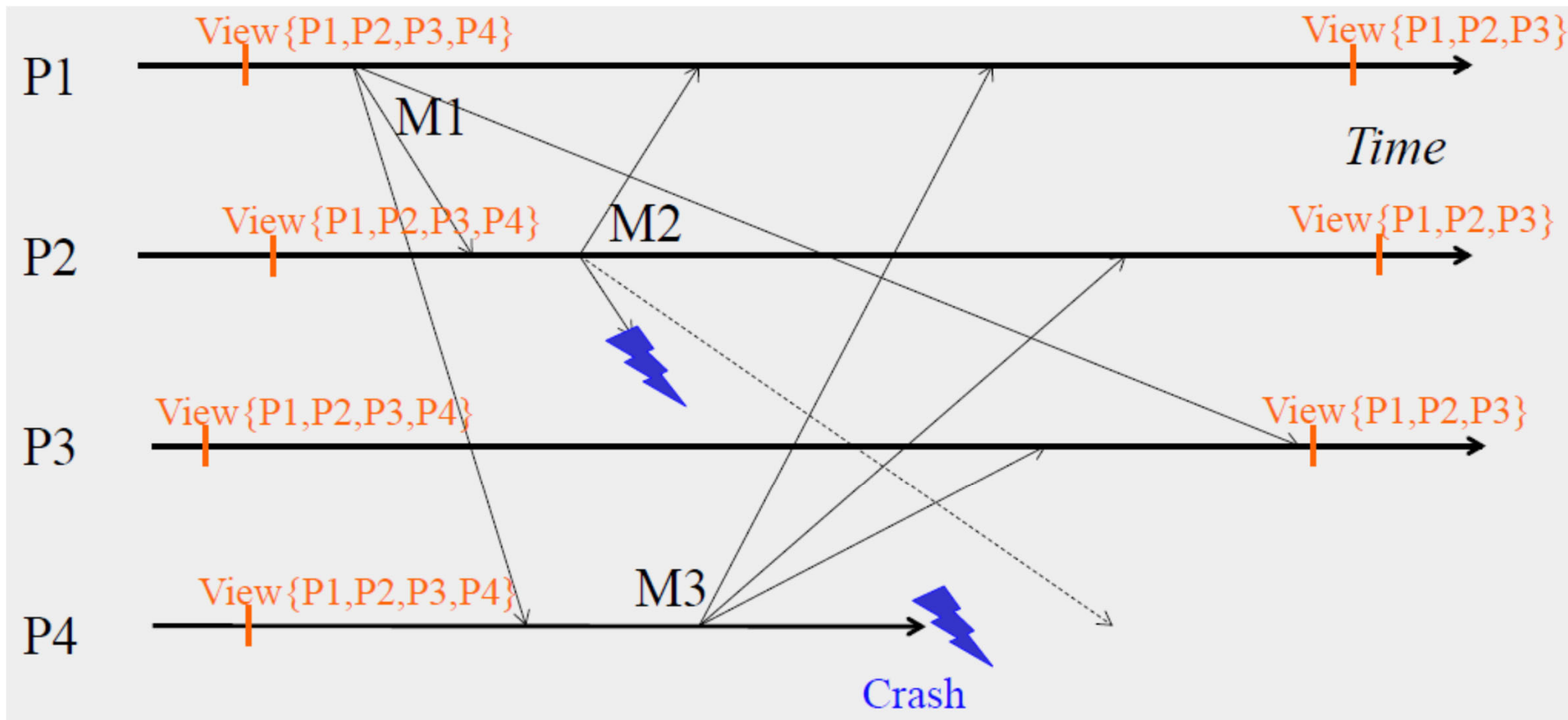


Acknowledgement: This example is adapted from Prof. Indranil Gupta of UIUC

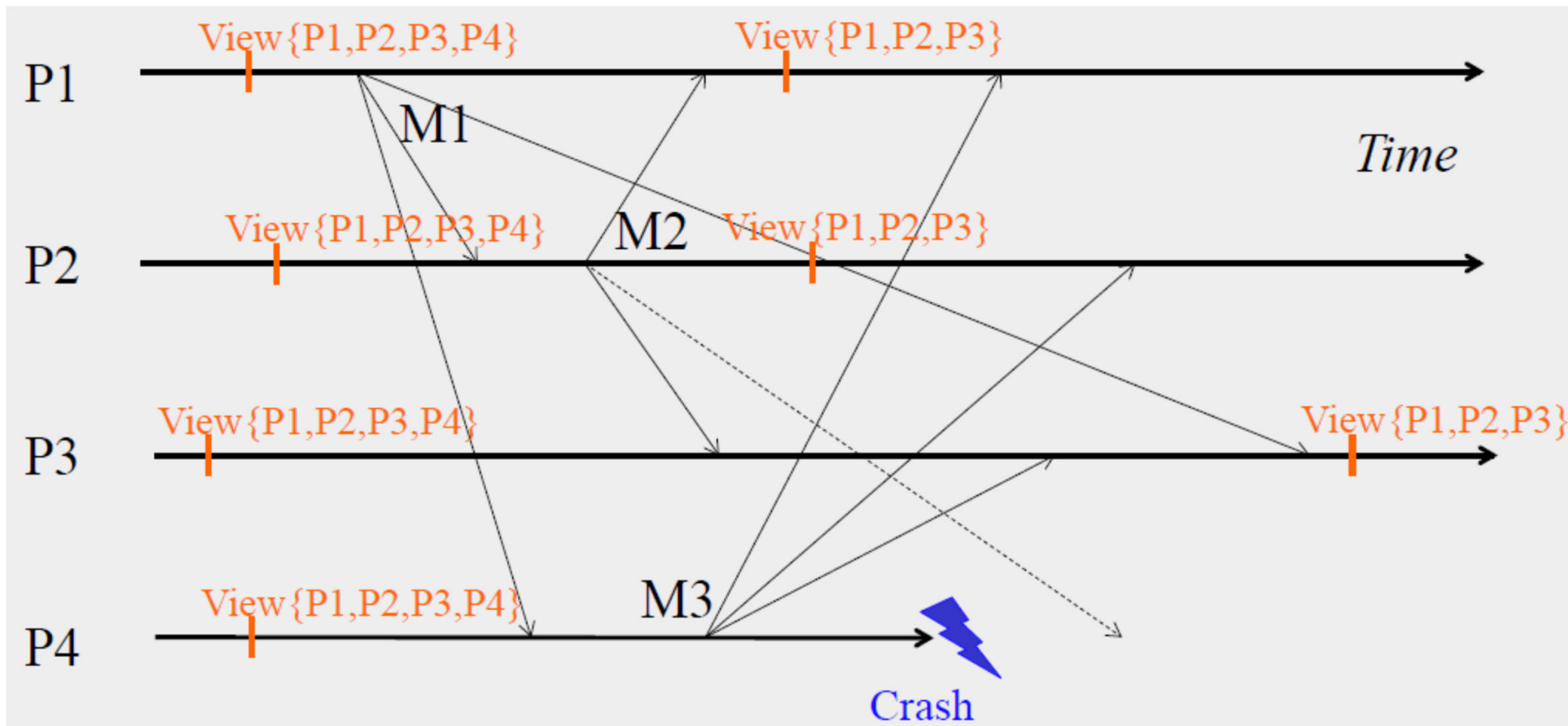
examples



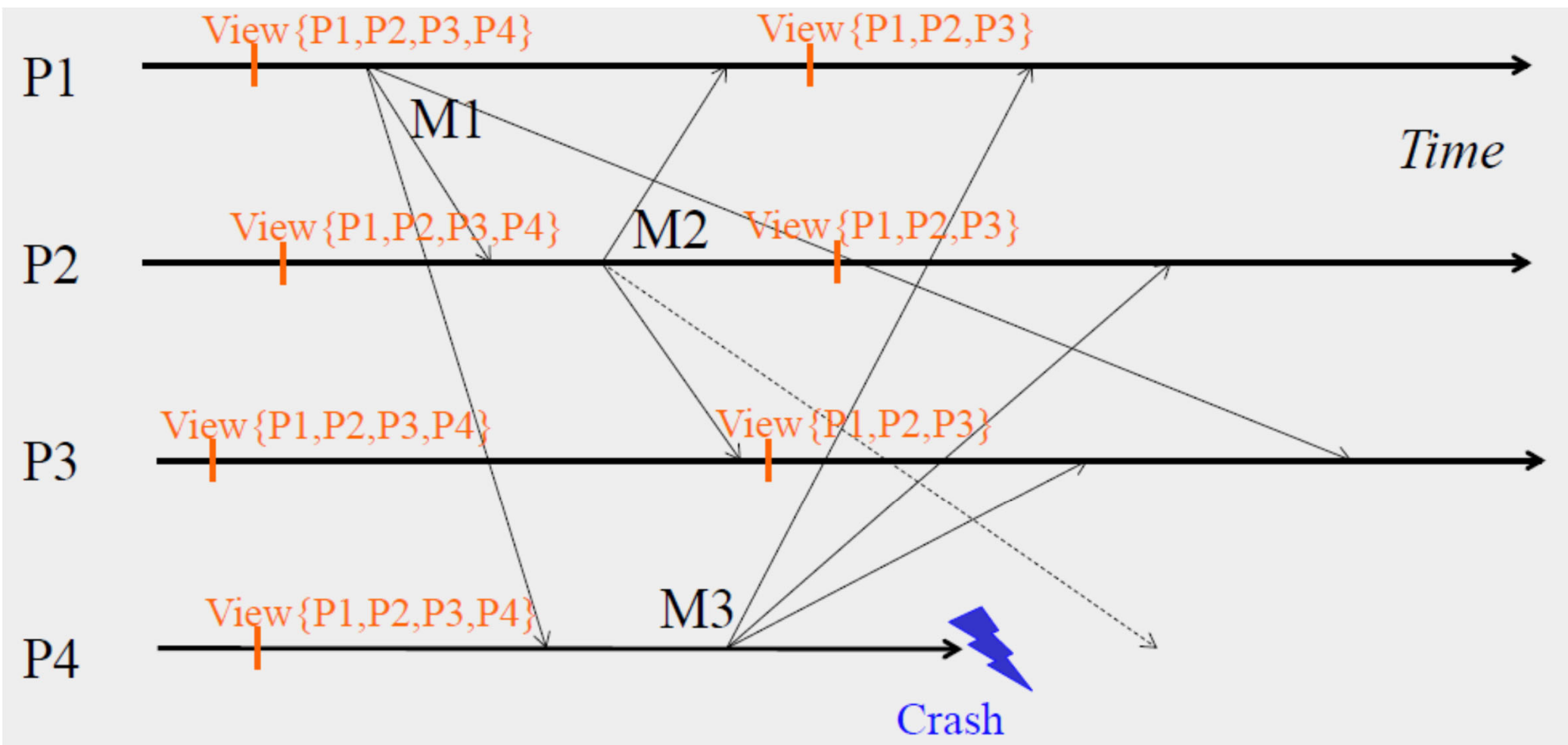
Acknowledgement: This example is adapted from Prof. Indranil Gupta of UIUC



Acknowledgement: This example is adapted from Prof. Indranil Gupta of UIUC



Acknowledgement: This example is adapted from Prof. Indranil Gupta of UIUC



Acknowledgement: This example is adapted from Prof. Indranil Gupta of UIUC

Implementing Virtual Synchrony

Assumptions

- ❑ A message is multicast using **reliable point-to-point** communication (e.g. TCP).
 - Although each point-to-point transmission is guaranteed to succeed, there are no guarantees that all group members receive the message
 - The sender might crash during or before transmitting the message
- ❑ **No process crash** during view change
- ❑ There is **one leader** for handling view change

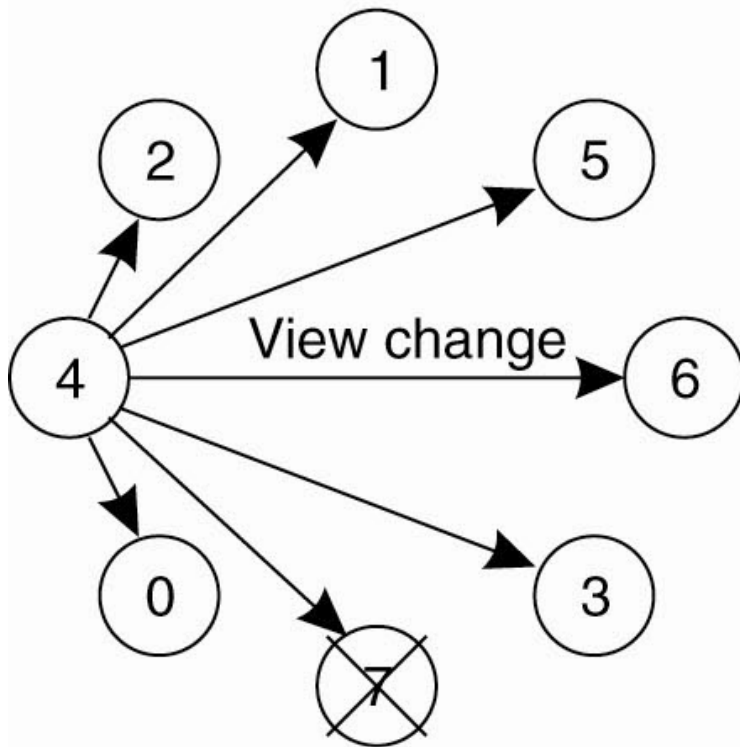
Problem

- ❑ Guarantee that all messages sent to group view G are delivered to all nonfaulty processes in G before the group view changes to G' (i.e. $G \neq G'$)

Solution

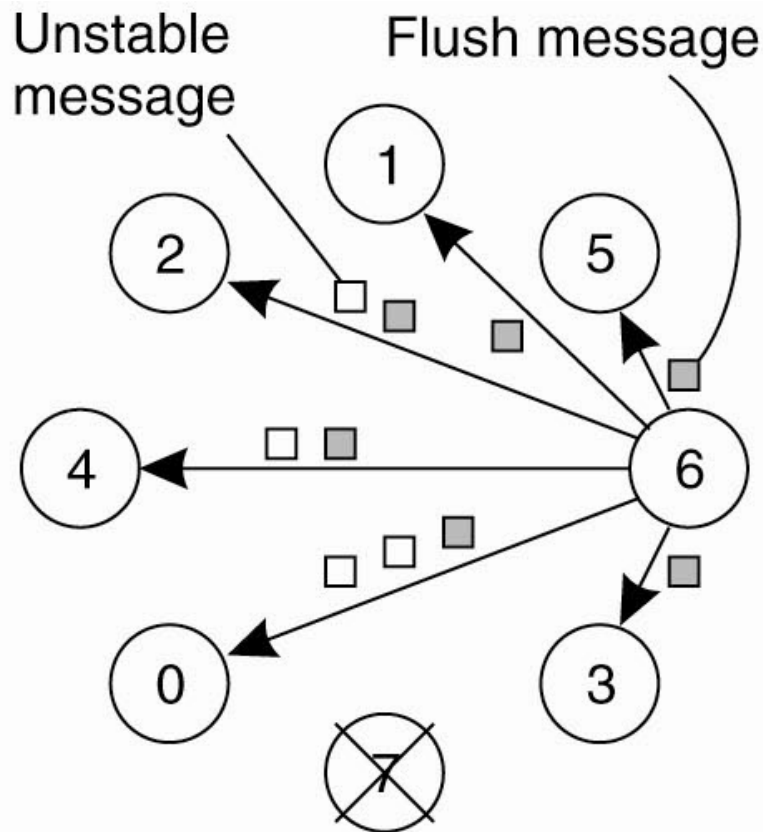
- ❑ For each message m , each process in G **keeps m** until it knows that all members in G have received m
- ❑ If m has been received by all members in G , m is called **stable**.
- ❑ Only stable messages can be deleted.

- A view change message is sent to all the processes in group view G when
 - A process wants to **join or leave** the group
 - A process detected that **failure** of another process



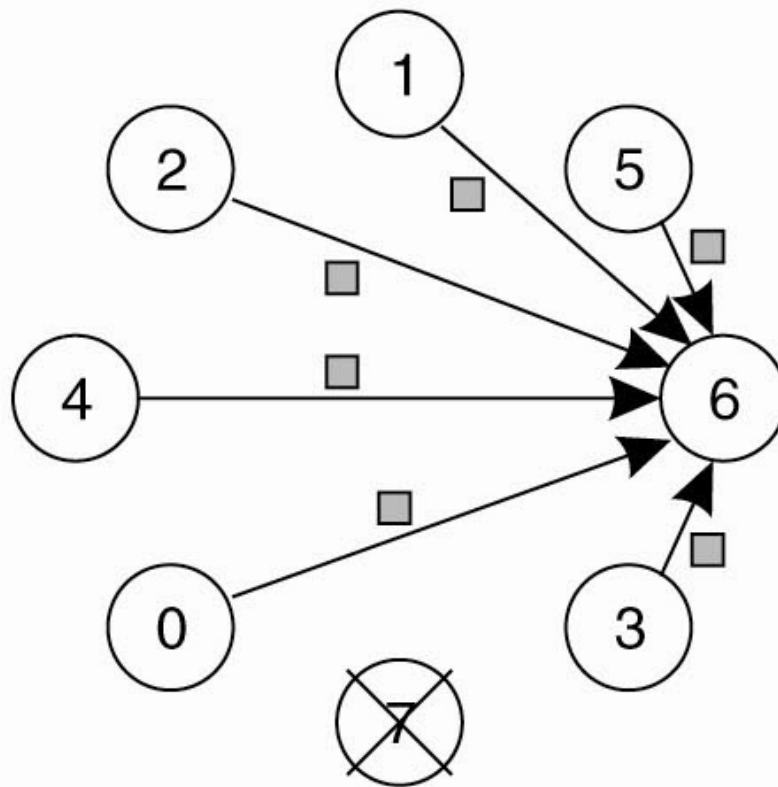
- Process 4 noticed the failure of process 7 and sends a view change message

- When a process P receives a view change message, P sends a copy of **any unstable message** from the current view to every process in the current view and marked these messages as stable
- To indicate that P no longer has any unstable messages and that it is prepared to install the new view, P multicast a **flush message** for installing the new view



- Process 6 sends out all its unstable messages followed by a flush message

- After P has received a flush message for the new view from each other process, P can install the new view



- Process 6 installs the new view when it has received a flush message from all other processes

reviews

- ❑ Understand the meaning of FIFO, total and causal order multicast and use an example to show why they are important for applications.
- ❑ Understand the three algorithms for implementing the three ordered multicast protocols.
- ❑ Can the causal order protocol be used to ensure FIFO order multicast? Explain why.
- ❑ Can the FIFO order multicast protocol be used to ensure total order multicast? Explain why.
- ❑ Design an algorithm that uses the partially ordered logical timestamp to ensure the casual-ordering of multicast messages.
- ❑ What properties does virtual synchrony have?

reviews

- ❑ The implementation of the virtual synchrony discussed in our lecture assumes that no failure occurs during the view change. Augment the scheme so that it can cope with failures during the view change.
- ❑ Assume that multiple processes can contact different processes in the process group concurrently for joining the process group. Augment the scheme discussed in our lecture to handle the view change correctly.
- ❑ Extend the implementation of the virtual synchrony discussed in our lecture to provide reliable total-order multicast.

readings

- ❑ K. Birman and T. Joseph. Reliable communications in the presence of failures. *ACM Trans. Comput. Syst.*, 5(1), 1987
- ❑ K. Birman, A. Schiper, and P. Stephenson. Lightweight causal and atomic group multicast. *ACM Transactions on Computer Systems*, 9(3):272--314, 1991